

A Formal Comparison Between Chain of Thought and Latent Thought

Kevin Xu¹ Issei Sato¹

Abstract

Chain of thought (CoT) elicits reasoning in large language models by explicitly generating intermediate tokens. In contrast, latent thought reasoning operates directly in the continuous latent space, enabling computation beyond discrete linguistic representations. While both approaches exploit iterative computation, their comparative capabilities remain underexplored. In this work, we present a formal analysis showing that latent thought admits more efficient parallel computation than inherently sequential CoT. In contrast, CoT enables approximate counting and sampling through stochastic decoding. These separations suggest the tasks for which depth-driven recursion is more suitable, thereby offering practical guidance for choosing between reasoning paradigms. Code is available at <https://github.com/kevin671/cot-vs-loop>.

1. Introduction

Transformer-based large language models (LLMs) (Vaswani et al., 2017) have shown strong performance across diverse tasks and have recently been extended to complex reasoning. Rather than directly predicting final answers, generating intermediate reasoning steps, known as chain of thought (CoT) (Wei et al., 2022), enhances reasoning abilities. This naturally raises the question: *why is CoT effective for complex tasks?* Recent studies have approached this question by framing reasoning as a computational problem and analyzing its complexity (Feng et al., 2023; Merrill & Sabharwal, 2024; Li et al., 2024; Nowak et al., 2024), showing that CoT improves performance by increasing the model’s effective depth through iterative computation, thereby enabling the solution of problems that would otherwise be infeasible.

As an alternative to CoT, recent work has explored latent

¹Department of Computer Science, The University of Tokyo, Japan. Correspondence to: Kevin Xu <kevinxu@g.ecc.u-tokyo.ac.jp>, Issei Sato <sato@g.ecc.u-tokyo.ac.jp>.

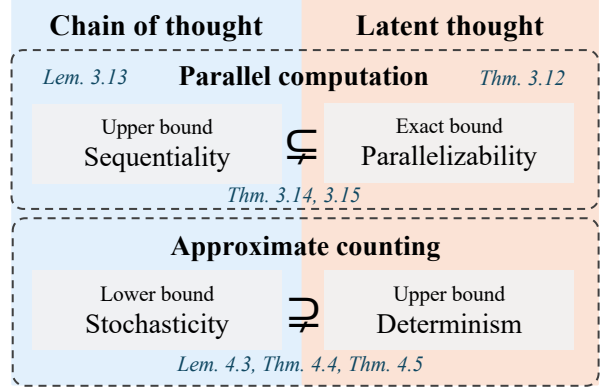


Figure 1. Overview of our formal comparison of the expressive power of chain-of-thought and latent thought reasoning.

thought, which reasons directly in the hidden state space rather than in the discrete token space. This paradigm includes chain of continuous thought (Coconut) (Hao et al., 2025), which replaces next tokens with hidden state, and *looped Transformer (looped TF)*, in which output hidden states are iteratively fed back as inputs (Dehghani et al., 2019). Such iterative architectures have been shown to enhance expressivity: Coconut enables the simultaneous exploration of multiple traces (Zhu et al., 2025a; Gozeten et al., 2025), while looped TF satisfies universality (Giannou et al., 2023; Xu & Sato, 2025) and demonstrates competitive empirical performance (Csordás et al., 2024; Bae et al., 2025; Zhu et al., 2025b).

These reasoning paradigms share the core idea of iteratively applying Transformers to enhance expressive power, which naturally leads to a fundamental question:

What is the separation between chain of thought and latent thought?

Recent studies characterize how expressivity scales with the number of iterations. Specifically, it has been shown that looped TF subsumes deterministic CoT (Saunshi et al., 2025), and exhibits a strict separation with only a logarithmic number of iterations (Merrill & Sabharwal, 2025a). Nevertheless, fundamental questions remain open:

*Does the separation extend beyond the logarithmic regime?
Is latent thought always more expressive than CoT?*

1.1. Our Contributions

In this work, we address both questions by clarifying the respective strengths and limitations of the two reasoning paradigms through a formal complexity-theoretic analysis of their expressive power. Specifically, we show that latent thought gains efficiency from its parallelizability, yielding separations beyond the polylogarithmic regime. In contrast, CoT benefits from stochasticity, which enables approximate counting. An overview is given in Fig. 1.

Latent thought enables parallel reasoning. By formalizing decision problems as the evaluation of directed acyclic graphs (DAGs), we reveal the parallel computational capability of latent thought utilizing continuous hidden states. This analysis can be formalized by relating the class of decision problems realizable by the model to Boolean circuits. In particular, Boolean circuits composed of logic gates such as AND, OR, NOT, and Majority, with polylogarithmic depth $\log^k n$ for $k \in \mathbb{N}$ and input size n , define the class TC^k , a canonical model of parallel computation. Circuit complexity plays a central role in analyzing the computational power of Transformer models: fixed-depth Transformers without CoT are known to be upper-bounded by TC^0 (Merrill & Sabharwal, 2023), and subsequent studies analyze how the expressivity of CoT scales their computational power in terms of Boolean circuit complexity (Li et al., 2024). We show that latent thought with $\log^k n$ iterations exactly captures the power of TC^k (Thm. 3.12); in contrast, CoT with $\log^k n$ steps cannot realize the full power of TC^k (Thm. 3.13) due to its inherent sequentiality. This yields a strict separation in favor of latent thought in polylogarithmic regime (Thm. 3.15), showing its efficiency in terms of the required number of iterations.

CoT enables approximate counting. A counting problem is a fundamental task in mathematics and computer science that determines the number of solutions satisfying a given set of constraints, including satisfying assignments of Boolean formulas, graph colorings, and partition functions (Arora & Barak, 2009). While exact counting for the complexity class $\#\text{P}$ is generally computationally intractable, approximation provides a feasible alternative. We show that CoT supports fully polynomial-time randomized approximation schemes (FPRAS), yielding reliable estimates even in cases where exact counting via deterministic latent thought reasoning is intractable (Lemma 4.3). Furthermore, leveraging classical results connecting approximate counting and sampling (Jerrum et al., 1986), we extend this separation to distribution modeling: there exist target distributions that CoT can approximately represent and sample from, but that remain inaccessible to latent thought (Theorem 4.4). To the best of our knowledge, this constitutes the first formal separation in favor of CoT.

2. Background

2.1. Models of Computation

We define a class of reasoning paradigms in which a Transformer block (Vaswani et al., 2017) is applied iteratively. Informally, CoT generates intermediate reasoning steps explicitly as tokens in an autoregressive manner. Formal definitions and illustrations are given in Appendix A.

Definition 2.1 (CoT, following Merrill & Sabharwal (2024)). Let $\mathcal{V}$ be a vocabulary, and let $\text{TF}_{\text{dec}} : \mathcal{V}^* \rightarrow \mathcal{V}$ denote an decoder-only Transformer. Given an input sequence $x = (x_1, \dots, x_n) \in \mathcal{V}^n$, the outputs of CoT are defined by

$$f_{\text{cot}}^0(x) := x, \quad f_{\text{cot}}^{k+1}(x) := f_{\text{cot}}^k(x) \cdot \text{TF}_{\text{dec}}(f_{\text{cot}}^k(x)),$$

where $\cdot$ denotes concatenation. We define the output to be the last tokens of $f_{\text{cot}}^{T(n)}(x) \in \mathcal{V}^{n+T(n)}$.

Coconut feeds the final hidden state as the embedding of the next token. Although the original Coconut model (Hao et al., 2025) can generate both language tokens and hidden states, we focus exclusively on hidden state reasoning steps, in order to compare its representational power with that of CoT. Here, $\mathbb{F}$ denotes the set of finite-precision floating-point numbers, and $d \in \mathbb{N}$ denotes the embedding dimension.

Definition 2.2 (Coconut). Let $\mathcal{V}$ be a vocabulary and let $\text{TF}_{\text{dec}}^{\text{Coconut}} : \mathcal{V}^* \times (\mathbb{F}^d)^* \rightarrow \mathbb{F}^d$ be a decoder-only Transformer that maps a fixed token prefix together with a hidden state to the next hidden state. Given an input sequence $x = (x_1, \dots, x_n) \in \mathcal{V}^n$, we define the hidden states recursively by

$$h^0 := (e(x_i))_{i=1}^n, \quad h^{k+1} := \text{TF}_{\text{dec}}^{\text{Coconut}}(x, h^k),$$

where $e : \mathcal{V} \rightarrow \mathbb{F}^d$ denotes an embedding. The output after $T(n)$ steps is obtained by decoding a suffix of the hidden state sequence ending at $h^{T(n)}$.

Looped TFs, by contrast, feed the entire model output back into the input without generating explicit tokens, recomputing all hidden states of the sequence at every iteration.

Definition 2.3 (Looped TF). Let $\text{TF} : \mathbb{F}^{d \times *} \rightarrow \mathbb{F}^{d \times *}$ denote a Transformer block. Given an input sequence $x = (x_1, \dots, x_n) \in \mathcal{V}^n$, the outputs are defined recursively by

$$f_{\text{loop}}^0(x) := (e(x_i))_{i=1}^n, \quad f_{\text{loop}}^{k+1}(x) := \text{TF}(f_{\text{loop}}^k(x)),$$

where $e : \mathcal{V} \rightarrow \mathbb{F}^d$ denotes an embedding. The output after $T(n)$ loop iterations is the decoded last tokens of $f_{\text{loop}}^{T(n)}(x)$.

Here, we assume that the input for looped TF may include sufficient padding so that its length is always at least as large as the output length, as in (Merrill & Sabharwal, 2025b). The definitions of the models describe their core architectures; the specific details may vary depending on the tasks to which they are applied.

Table 1. Comparison between prior theoretical analyses and our work on the computational power of CoT, Coconut, and looped TF.

Paper	Model	Det	Pro	Class	Problem Setting
Li et al. (2024)	CoT	✓	-	✓	Boolean circuit
Nowak et al. (2024)	CoT	-	✓	✓	Language modeling
Saunshi et al. (2025)	CoT / Loop	✓	-	-	Group composition
Merrill & Sabharwal (2025b)	CoT / Loop	✓	-	✓	Uniform TC ^k
Gozeten et al. (2025); Zhu et al. (2025a)	CoT / Coconut	✓	-	-	Graph exploration
Svete & Sabharwal (2025)	CoT / Loop / Masked	✓	-	✓	Uniform TC ^k
Ours	CoT / Coconut / Loop	✓	✓	✓	TC ^k & Approximate counting

2.2. Related Work

To understand the expressive power of Transformers, previous work studies which classes of problems can be solved and with what computational efficiency. These questions can be naturally analyzed within the framework of computational complexity theory. Such studies on CoT and latent thought are summarized below and in Table 1.

Computational power of chain of thought. The expressivity of Transformers is limited by bounded depth (Merrill & Sabharwal, 2023), whereas CoT enhances their expressiveness by effectively increasing the number of sequential computational steps, enabling the solution of problems that would otherwise be intractable for fixed-depth architectures (Feng et al., 2023). Recent work has investigated how the expressivity of CoT scales with the number of reasoning steps, formalizing CoT for decision problems and computational complexity classes (Merrill & Sabharwal, 2024; Li et al., 2024). Beyond decision problems, CoT has been further formalized in a probabilistic setting for representing probability distributions over strings (Nowak et al., 2024).

Computational power of latent thought. Latent thought is an alternative paradigm for increasing the number of computational steps without being constrained to the language space, with the potential to enhance model expressivity. In particular, Coconut has been shown to enable the simultaneous exploration of multiple candidate reasoning traces (Zhu et al., 2025a; Gozeten et al., 2025). Looped TFs can simulate iterative algorithms (Yang et al., 2024; de Luca & Fountoulakis, 2024) and, more generally, realize polynomial-time computations (Giannou et al., 2023). Recent results further demonstrate advantages over chain of thought reasoning: looped TFs can subsume the class of deterministic computations realizable by CoT using the same number of iterations (Saunshi et al., 2025), and exhibit a strict separation within the same logarithmic iterations (Merrill & Sabharwal, 2025a). Concurrent work (Zhu et al., 2025a; Merrill & Sabharwal, 2025b) has also identified connections between parallel reasoning and Coconut and diffusion models.

3. Latent Thought Enables Parallel Reasoning

We formalize the reasoning problem as a graph evaluation problem. Section 3.2 illustrates how each model approaches the same problem differently, providing intuitive insight into their contrasting capabilities. Building on these observations, Section 3.3 characterizes their expressive power and establishes a formal separation between them.

3.1. Problem Setting

Reasoning problems that can be solved by straight-line programs admit representations as directed acyclic graphs (DAGs) (Aho & Ullman, 1972), as illustrated in Fig. 2(a).

Definition 3.1 (Computation graph). Let Σ be a finite alphabet, and let $\mathcal{F}$ denote a finite set of functions $f : \Sigma^* \rightarrow \Sigma$. A *computation graph* is a directed acyclic graph $G_n = (V_n, E_n)$ that defines a function $F_{G_n} : \Sigma^n \rightarrow \Sigma^{m(n)}$, where $m(n)$ denotes the output length. Here V_n denotes the set of nodes, consisting of (i) n input nodes with in-degree 0, (ii) function nodes labeled by $f \in \mathcal{F}$, which take as arguments the predecessor nodes specified by their incoming edges in E_n , and (iii) $m(n)$ output nodes with out-degree 0. The overall function is obtained by evaluating the graph in topological order. The *size* of the graph is $|V_n|$, denoted by $\text{size}(G_n)$, and its *depth* is the length of the longest path from an input to an output node, denoted by $\text{depth}(G_n)$.

Assumptions on models. Our goal is to evaluate the computational efficiency of each model via an asymptotic analysis of how the required number of reasoning steps or loops scales with the input size n . Beyond *time complexity*, we also allow the *space complexity* of the model to scale with the input size n . In particular, the embedding dimension in Transformer blocks can be viewed as analogous to the number of processors in classical parallel computation models. Accordingly, we adopt a *non-uniform* computational model, in which a different model is allowed for each input size. This non-uniform setting is standard in the study of circuit complexity and parallel computation (Cook, 1985), and is consistent with prior analyses of Transformers and CoT (Sanford et al., 2024b; Li et al., 2024).

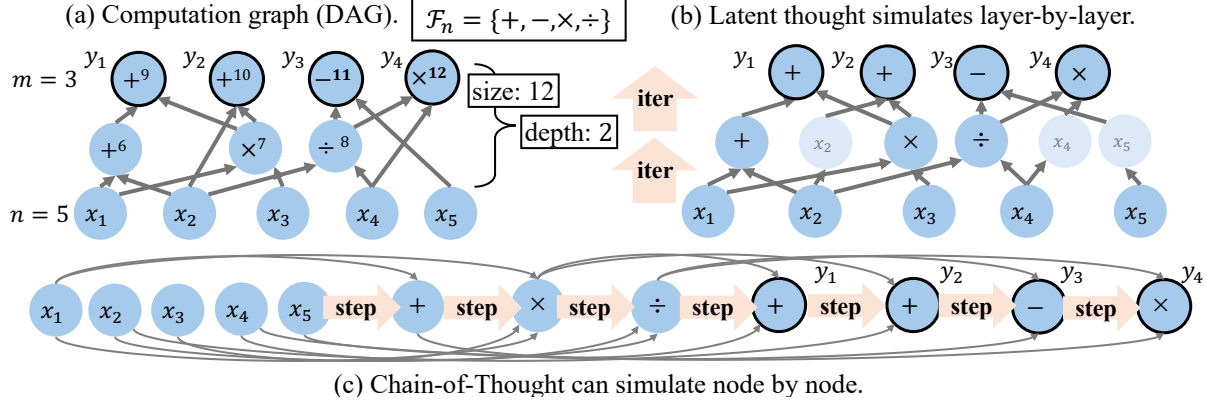


Figure 2. Comparison of reasoning paradigms for evaluating a DAG. (a) A computation graph G_n . (b) Latent thought can simulate the computation layer by layer in parallel, using a number of loops equal to the depth of the graph, $\text{depth}(G_n)$. (c) CoT can sequentially simulate the computation node by node, using a number of steps proportional to the size of the graph, $O(\text{size}(G_n))$.

On the fairness of comparing steps and loops. We analyze expressivity in terms of the number of reasoning steps. Although this may appear unfair in terms of raw computation, it is justified when comparing latency. Specifically, CoT benefits from KV caching, which makes each step computationally inexpensive; however, accessing cached states is typically memory-bound, leaving compute resources underutilized. In contrast, looped TFs recompute over the full sequence at each iteration, incurring higher arithmetic cost but achieving higher arithmetic intensity and better utilization of modern parallel hardware. As a result, the latency of looped TFs is comparable to that of CoT.

3.2. CoT Suffices with Size-scaled Steps and Latent Thought Suffices with Depth-scaled Iterations

We show how each model can evaluate DAGs, which provides a lower bound on their expressivity and offers intuition for the distinctions between the models, in terms of sequentiality and parallelizability. Before presenting our main result, we first state the underlying assumptions.

Definition 3.2 (Merrill & Sabharwal, 2023). The model is *log-precision*, where each scalar is stored with $O(\log n)$ bits and every arithmetic operation is rounded to that precision.

Assumption 3.3 (Poly-size graph). $\text{size}(G_n) \in \text{poly}(n)$.

Assumption 3.4 (Poly-efficient approximation, cf. (Feng et al., 2023)). Each node function of G_n can be approximated by a log-precision feedforward network whose parameter size is polynomial in the input length and the inverse of the approximation error. We denote by $\text{ff_param}(G_n)$ an upper bound such that every $f \in \mathcal{F}$ admits such a network with at most $\text{ff_param}(G_n)$ parameters.

Under these assumptions, we show that CoT can simulate computation by sequentially decoding nodes, where intermediate tokens serve as a scratchpad allowing the evaluation of each node once all its predecessors have been computed.

Theorem 3.5 (CoT for DAGs). *Let $\{G_n\}_{n \in \mathbb{N}}$ be a family of computation graphs that satisfy Assumptions 3.3 and 3.4. Then, for each $n \in \mathbb{N}$, there exists a log-precision CoT with parameter size bounded by $O(\text{ff_param}(G_n))$, such that for every input $x \in \Sigma^n$, the model outputs $F_{G_n}(x)$ with steps proportional to the “size” of the graph, i.e., $O(\text{size}(G_n))$.*

Proof sketch. At each step, the attention layer retrieves the outputs of predecessor nodes from previously generated tokens, and a feed-forward layer then computes the node function, whose output is generated as the next token. $\square$

In contrast, latent thought can operate in parallel, layer by layer, where all nodes at the same depth are computed simultaneously, provided that the model has sufficient size.

Theorem 3.6 (Latent thought for DAGs). *Let $\{G_n\}_{n \in \mathbb{N}}$ be a family of computation graphs that satisfy Assumptions 3.3 and 3.4. Then, for each $n \in \mathbb{N}$, there exists a log-precision Coconut and looped TF with parameter size $O(\text{ff_param}(G_n) \cdot \text{size}(G_n))$, such that for every input $x \in \Sigma^n$, it computes $F_{G_n}(x)$ with iterations proportional to the “depth” of the graph G_n , i.e., $O(\text{depth}(G_n))$.*

Proof sketch. The role assignment of each layer is based on (Li et al., 2024), as shown in Figure 9. An attention layer aggregates its inputs into a single hidden state. Unlike discrete tokens, continuous latent states allow the simultaneous encoding of the outputs of multiple nodes, enabling the feed-forward layer to compute node functions in parallel. $\square$

Remark. Illustrations are provided in Fig. 2, with formal proofs deferred to Appendix B. These results reveal distinct characteristics: CoT can utilize intermediate steps as scratchpad memory and perform computations sequentially, whereas latent thought can leverage structural parallelism to achieve greater efficiency with sufficient resources.

3.3. Separation in Polylogarithmic Iterations

In this section, we shift to formal decision problems to precisely characterize the computational power of each reasoning paradigm, clarify what cannot be achieved, and use these limitations to derive rigorous separations. We begin by defining their complexity classes, as in (Li et al., 2024).

Definition 3.7 (Complexity Classes CoT, CT and Loop). Let $\text{CoT}[T(n), d(n), s(n)]$, $\text{CT}[T(n), d(n), s(n)]$, and $\text{Loop}[T(n), d(n), s(n)]$ denote the sets of languages $\mathcal{L} : \{0, 1\}^* \rightarrow \{0, 1\}$ for which there exists a *deterministic* CoT, Coconut, and looped TF, respectively, denoted by M_n for each input size n , with embedding size $O(d(n))$ and $O(s(n))$ bits of precision, such that for all $x \in \{0, 1\}^n$, the final output token after $O(T(n))$ iterations equals $\mathcal{L}(x)$.

Boolean circuits serve as a standard formal model of computation, where processes are defined by the evaluation of DAGs with well-established complexity measures.

Definition 3.8 (Informal). A *Boolean circuit* is a DAG over the alphabet $\Sigma = \{0, 1\}$, where each internal node (gate) computes a Boolean function such as AND, OR, or NOT. $\text{SIZE}[s(n)]$ and $\text{DEPTH}[d(n)]$ denote the class of languages decidable by a non-uniform circuit family $\{C_n\}$ with size $O(s(n))$ and depth $O(d(n))$, respectively.

First, we formalize the results of the previous section to show that latent thought iterations can represent circuit depth, whereas CoT corresponds to circuit size.

Theorem 3.9 (Li et al., 2024). $\forall T(n) \in \text{poly}(n)$,

$$\text{SIZE}[T(n)] \subseteq \text{CoT}[T(n), \log n, 1].$$

Theorem 3.10. For any function $T(n) \in \text{poly}(n)$ and any non-uniform circuit family $\{C_n\}$, it holds that

$$\begin{aligned} \text{DEPTH}[T(n)] &\subseteq \text{Loop}[T(n), \text{size}(C_n), 1], \\ \text{DEPTH}[T(n)] &\subseteq \text{CT}[T(n), \text{size}(C_n), 1]. \end{aligned}$$

Boolean circuits serve as a formal model of parallel computations that run in polylogarithmic time using a polynomial number of processors (Stockmeyer & Vishkin, 1984).

Definition 3.11. For each $k \in \mathbb{N}$, the classes NC^k , AC^k , and TC^k consist of languages decidable by non-uniform circuit families of size $\text{poly}(n)$ and depth $O(\log^k n)$, using bounded-fanin Boolean gates, unbounded-fanin AND/OR gates, and threshold gates, respectively.

We then characterize the exact computational power of latent thought in the parallel computation regime.

Theorem 3.12. For each $k \in \mathbb{N}$, it holds that

$$\begin{aligned} &\text{Loop}[\log^k n, \text{poly}(n), 1 \text{ (resp. } \log n)] \\ &= \text{CT}[\log^k n, \text{poly}(n), 1 \text{ (resp. } \log n)] \\ &= \text{AC}^k \text{ (resp. } \text{TC}^k). \end{aligned}$$

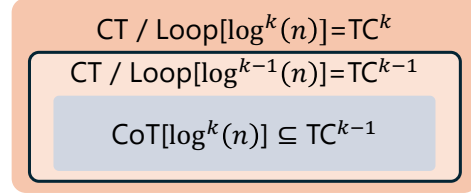


Figure 3. The separation between latent thought and CoT for decision problems, under polylogarithmic iterations.

Proof sketch. The inclusion from circuits to latent thought follows from Theorem 3.10. For the converse inclusion, we build on the arguments of prior work (Merrill & Sabharwal, 2023; Li et al., 2024), which show that a fixed-depth Transformer block under finite precision is contained in AC^0 (or TC^0 under logarithmic precision). We extend their analysis to the looped setting, which can be unrolled into a TC^k circuit by composing a TC^0 block for $\log^k n$ iterations. $\square$

Moreover, we establish an upper bound on the power of CoT in the parallel computation regime. This limitation arises from the inherently sequential nature of CoT.

Lemma 3.13. For each $k \in \mathbb{N}$, it holds that

$$\text{CoT}[\log^k n, \text{poly}(n), \log n] \subseteq \text{TC}^{k-1}.$$

Proof. The total $\log^k n$ steps can be divided into $\log^{k-1} n$ blocks, each consisting of $\log n$ steps. Since $\text{CoT}[\log n, \text{poly}(n), \log n] \subseteq \text{TC}^0$ (Li et al., 2024), each block with the previous block's outputs fed as inputs to the next block can be simulated in TC^0 ; iterating this over $\log^{k-1} n$ layers yields a circuit in TC^{k-1} . $\square$

These results lead to a separation in expressive power under standard complexity assumptions, as illustrated in Figure 3.

Theorem 3.14. For each $k \in \mathbb{N}$, if $\text{TC}^{k-1} \subsetneq \text{NC}^k$, then

$$\begin{aligned} \text{CoT}[\log^k n, \text{poly}(n), \log n] &\subsetneq \text{Loop}[\log^k n, \text{poly}(n), 1], \\ \text{CoT}[\log^k n, \text{poly}(n), \log n] &\subsetneq \text{CT}[\log^k n, \text{poly}(n), 1]. \end{aligned}$$

Theorem 3.15. For each $k \in \mathbb{N}$, if $\text{TC}^{k-1} \subsetneq \text{TC}^k$, then

$$\begin{aligned} \text{CoT}[\log^k n, \text{poly}(n), \log n] &\subsetneq \text{Loop}[\log^k n, \text{poly}(n), \log n], \\ \text{CoT}[\log^k n, \text{poly}(n), \log n] &\subsetneq \text{CT}[\log^k n, \text{poly}(n), \log n]. \end{aligned}$$

Remark. The claims follow directly from Theorem 3.12 and Lemma 3.13. The established separations of the complexity classes, as summarized in Fig. 3, show that latent thought reasoning enables efficient parallel solutions more effectively than CoT, which is inherently sequential.

4. CoT Enables Approximate Counting

In the previous section, we showed that for decision problems, latent thought can yield more efficient solutions than CoT. This naturally raises the question of whether latent thought is universally more powerful than CoT. While prior work has shown that CoT can be simulated by looped Transformer models for deterministic decision problems under deterministic decoding (Saunshi et al., 2025), we found that this result does not directly extend to probabilistic settings with stochastic decoding. Accordingly, we shift our focus from efficiency in terms of the number of reasoning steps to expressive capability under polynomially many iterations.

4.1. Preliminaries

Approximate counting. Formally, let Σ be a finite alphabet and let $R \subseteq \Sigma^* \times \Sigma^*$ be a relation. For an input $x \in \Sigma^*$, define $R(x) := \{y \in \Sigma^* \mid (x, y) \in R\}$, and the counting problem is to determine $|R(x)|$. A wide class of natural relations admits a recursive structure, which allows solutions to be constructed from smaller subproblems.

Definition 4.1 (Informal: Self-reducibility (Schnorr, 1976)). A relation R is *self-reducible* if there exists a polynomial-time procedure that, given any input x and prefix $y_{1:k}$ (with respect to a fixed output order), produces a subinstance $\psi(x, y_{1:k})$ such that every solution z of $\psi(x, y_{1:k})$ extends $y_{1:k}$ to a solution of $R(x)$ (and conversely), i.e., $R(\psi(x, y_{1:k})) = \{z \mid \text{concat}(y_{1:k}, z) \in R(x)\}$.

While exact counting is intractable, there exist efficient randomized approximation algorithms (Karp & Luby, 1983).

Definition 4.2 (FPRAS). An algorithm is called a *fully polynomial-time randomized approximation scheme* (FPRAS) for a function f if, for any $\varepsilon > 0$ and $\delta > 0$, it outputs an estimate $\hat{f}(x)$ such that

$$\Pr[(1 - \varepsilon)f(x) \leq \hat{f}(x) \leq (1 + \varepsilon)f(x)] \geq 1 - \delta,$$

and runs in time polynomial in $|x|$, $1/\varepsilon$, and $\log(1/\delta)$.

The class of counting problems that admit an FPRAS is denoted by FPRAS. Although randomized algorithms provide only probabilistic guarantees, they are often both more efficient and simpler than their deterministic counterparts, denoted by FPTAS (Definition C.11). For example, counting the number of satisfying assignments of a DNF formula admits an FPRAS based on Monte Carlo methods (Karp et al., 1989), whereas no FPTAS is known for this problem. Moreover, probabilistic analysis enables us to capture algorithmic behavior on typical instances arising in real-world applications (Mitzenmacher & Upfal, 2017).

Probabilistic models of computation. In contrast to the deterministic models considered in the previous section,

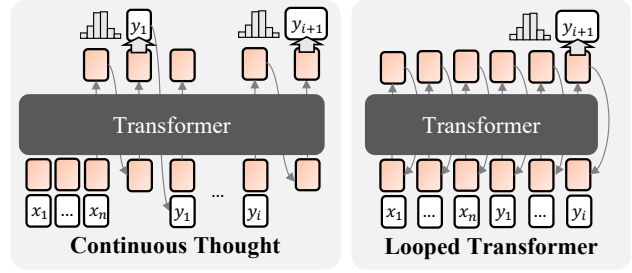


Figure 4. Probabilistic models of computation with latent thought. Each output token y_i is stochastically generated.

we now study probabilistic models that define a conditional distribution over output strings $y = (y_1, \dots, y_m) \in \Sigma^*$. We consider autoregressive next-token prediction of the form

$$p(y \mid x) = \prod_{i=1}^m p(y_i \mid x, y_{<i}),$$

where the model is allowed to perform additional reasoning steps before producing each output token y_i . This formulation was first used to formalize CoT for language modeling by Nowak et al. (2024). We also allow stochastic decoding for latent reasoning at the token level: reasoning iterations are performed entirely in hidden space and no linguistic tokens are sampled except for the output token y_i , as illustrated in Fig. 4. This definition is consistent with practical implementations (Csordás et al., 2024; Bae et al., 2025). Within this framework, we define complexity classes of probabilistic models, denoted by pCoT, pCT, and pLoop, respectively. Formal definitions are in Appendix C.1.

4.2. Separation in Approximate Counting

We first analyze the expressivity of the token-level conditional prediction at each step, $p(y_i \mid x, y_{<i})$, and show that CoT is strictly more expressive than latent thought in this setting. The key distinction is whether intermediate computation permits sampling. CoT explicitly samples intermediate reasoning tokens, inducing stochastic computation and enabling the emulation of randomized algorithms. In contrast, latent thought performs only deterministic transformations in latent space, resulting in deterministic computation.

Lemma 4.3 (Informal). Assume that $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations. There exists a self-reducible relation R and an associated function $f : \Sigma^* \times \Sigma^* \rightarrow \mathbb{N}$ defined by $f(x, y_{<i}) := |\{z \in \Sigma^* : (x, y_{<i}z) \in R\}|$ such that CoT with polynomially many steps admits an FPRAS for f . Whereas, no latent thought with polynomially many iterations admits the same approximation guarantee.

Proof sketch. For self-reducible relations, approximating the counting function f on subproblems is polynomial-time inter-reducible with approximating $|R(x)|$ (Jerrum et al.,

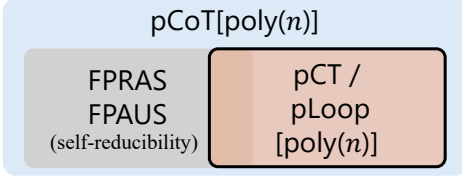


Figure 5. The separation for approximate counting (sampling).

1986). If latent thought with polynomially many iterations admitted an FPTAS for f , then it would induce a deterministic FPTAS for $|R(x)|$, contradicting the assumption. $\square$

4.3. Separation in Approximate Sampling

We then move from token-level conditional distributions $p(y_i | x, y_{<i})$ to the full sequence-level distribution $p(y | x)$. Beyond approximate counting, we establish a separation for approximate sampling problems. Specifically, we construct target distributions for which the complexity of each conditional can be reduced to approximate counting.

Theorem 4.4. *Assume that $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations. There exists a distribution $p(y | x)$ over $y \in \Sigma^*$ and $x \in \Sigma^n$ such that a CoT with a polynomial number of steps, whose induced output conditionals are denoted by $q(y_i | x, y_{<i})$, admits an FPRAS for approximating the conditional probabilities $p(y_i | x, y_{<i})$ for all $x \in \Sigma^n$, indices $i \geq 1$, and prefixes $y_{<i} := (y_1, \dots, y_{i-1})$. In contrast, no latent thought with polynomially many iterations admits the same approximation guarantee.*

Proof sketch. Define the target distribution p to be the uniform distribution supported on the solution set $R(x)$. We rely on the classical result that approximate sampling from the uniform distribution over solutions, captured by the class FPAUS, is polynomial-time inter-reducible with approximate counting for self-reducible relations (Jerrum et al., 1986). Let $U(\cdot | x)$ denote the uniform distribution over solutions of a self-reducible relation $R(x)$. This distribution admits an autoregressive factorization $U(y | x) = \prod_{i=1}^m p(y_i | x, y_{<i})$, where each conditional probability is given by $p(y_i | x, y_{<i}) = \frac{|\{z \in \Sigma^* : (x, y_{1:i-1}z) \in R\}|}{|\{z \in \Sigma^* : (x, y_{1:i}z) \in R\}|}$. We show that each conditional probability, expressed as a ratio of subproblem counts, reduces to approximate counting. Then, applying Lemma 4.3 to these conditionals yields the desired separation for approximate sampling. $\square$

Consequently, we obtain the following separations in favor of CoT, as also shown in Fig. 5.

Theorem 4.5. *Assuming $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations, it holds that*

$$\forall \mathcal{M} \in \{\text{pCT}, \text{pLoop}\}, \quad \mathcal{M}[\text{poly}(n)] \subsetneq \text{pCoT}[\text{poly}(n)].$$

5. Experiments

In this section, we provide empirical validation of our theoretical results on tasks with well-characterized complexity. Specifically, we study parallelizable tasks to empirically validate the efficiency of latent thought predicted in Section 3, and approximate counting and sampling tasks to demonstrate the effectiveness of CoT as shown in Section 4.

5.1. Experimental Setting

Fundamental algorithmic reasoning tasks. We use four problems. (1) Word problems for finite non-solvable groups: given a sequence of generators, the task is to evaluate their composition, which is NC^1 -complete (Barrington, 1986), also studied for Looped TF (Merrill & Sabharwal, 2025a). (2) s - t connectivity (STCON): given a directed graph $G = (V, E)$ and two vertices $s, t \in V$, the task is to decide whether t is reachable from s , which belongs to TC^1 (Gibbons & Rytter, 1989). (3) Arithmetic expression evaluation: given a formula consisting of $+$, $\times$, $-$, $/$ operations on integers, the task is to evaluate it. This problem is TC^0 -reducible to Boolean formula evaluation (Feng et al., 2023), which is NC^1 -complete (Buss, 1987). (4) Edit distance: given two strings x and y , the task is to compute the minimum cost to transform x into y . By reducing the dynamic programming formulation to shortest paths, this problem is in TC^1 (Apostolico et al., 1990).

Approximate counting tasks. We consider DNF counting and uniform sampling of graph colorings, both of which admit fully polynomial randomized approximation schemes for counting and sampling (FPRAS and FPAUS). Specifically, DNF counting admits an FPRAS via Monte Carlo sampling (Karp et al., 1989), while approximate counting and sampling of graph colorings admit an FPAUS based on rapidly mixing Markov chain Monte Carlo under suitable degree and color constraints (Jerrum, 1995).

Training strategy. Since our primary objective is to study expressive power, we allow flexibility in optimization and training strategies. For CoT models, training is performed with supervision from explicit sequential algorithms. For fewer CoT steps, we compare two strategies: uniformly selecting steps from the indices of the complete trajectory (Bavandpour et al., 2025), and stepwise internalization (distillation) methods (Deng et al., 2024). For latent thought, we observe that looped TF is easier to train than Coconut, and therefore adopt looped TF as our instantiation of latent thought, with curriculum learning applied to certain tasks.

5.2. Results

Table 2 reports results on parallelizable tasks, comparing latent thought and CoT under varying numbers of iterations.

Table 2. Accuracy (%) of CoT and looped TF on parallelizable tasks across different numbers of iterations. Here, n denotes the problem size. For CoT, we report the best accuracy achieved across the two training strategies.

Task	n	Looped Transformer				Chain of Thought			
		2	4	6	8	8	16	32	64
Word Problem	64	0.8	0.8	100.0	100.0	0.8	0.8	100.0	100.0
Graph Connectivity	32	80.8	95.8	99.0	99.0	81.0	81.4	88.2	100.0
Arithmetic Evaluation	32/16	43.7	99.4	99.5	99.7	47.3	47.6	48.2	82.5
Edit Distance	32/16	57.3	72.9	86.2	90.7	76.5	80.9	87.5	94.8

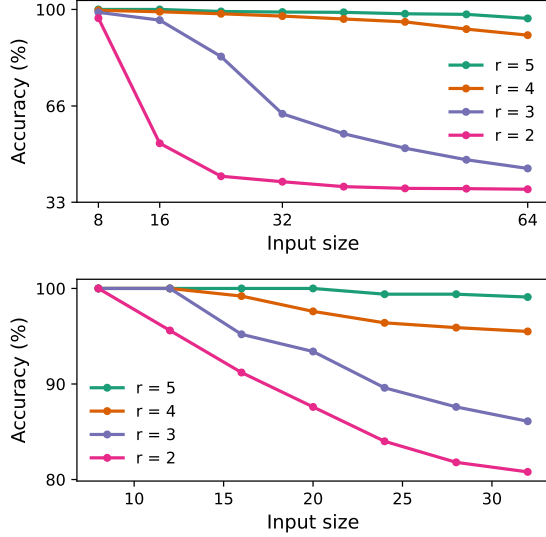


Figure 6. Accuracy of looped TFs on the arithmetic evaluation (top) and the connectivity (bottom). Each curve shows the performance for a fixed loop count r as the input size n increases.

Latent thought solves the problems with fewer iterations than CoT requires to reach comparable performance. These empirical results are consistent with our theoretical analysis: latent thought supports efficient parallel reasoning, in contrast to the inherently sequential nature of CoT. We also evaluate the relationship between performance, input size, and the number of iterations, as in prior studies (Sanford et al., 2024b; Merrill & Sabharwal, 2025a). Figure 6 presents our results for looped TFs, illustrating that as the input size n increases, the number of loops required to maintain high accuracy grows only logarithmically, supporting our theoretical claim in the (poly-)logarithmic regime.

Figure 7 shows the results on the approximate counting or sampling tasks. For approximate counting, CoT performs Monte Carlo estimation: the effective number of samples is given by the product of the number of reasoning steps per trial and the number of independent trials. The probability mass plot illustrates how the empirical distribution over valid colorings compares to the target uniform distribution. We observe that CoT produces a distribution that is closer to

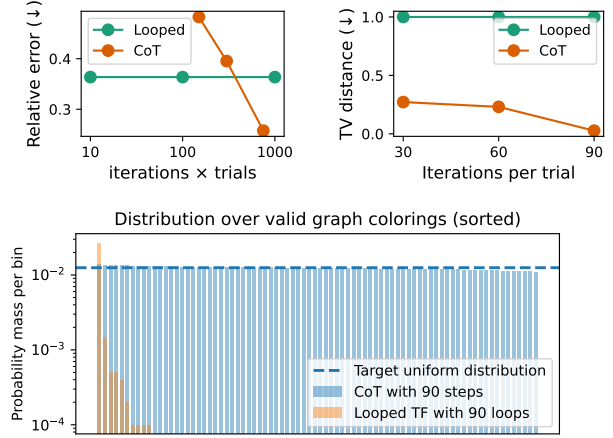


Figure 7. Top: Relative error for counting (left) and TV distance to uniform over valid colorings for sampling (right). Bottom: Empirical distributions for approximate sampling of graph colorings.

uniform, whereas the looped model concentrates probability mass on a smaller subset of solutions. This indicates that CoT achieves more uniform coverage of the valid colorings.

6. Conclusion

We formally analyze the computational capabilities of chain-of-thought and latent thought reasoning, providing a rigorous comparison that reveals their respective strengths and limitations. Specifically, we show that latent thought enables efficient parallel computation, whereas CoT enables randomized approximate counting. Our results provide practical guidance for selecting between reasoning paradigms: latent reasoning is more suitable for problems that can be solved efficiently, whereas CoT is more effective for more complex problems. For future work, an important direction is to investigate whether techniques such as distillation can reduce the number of iterations without compromising computational power. Another promising avenue is to extend our analysis to diffusion language models, which possess both parallelizability and stochasticity. Moreover, extending the analysis to realistic downstream tasks remains an important direction.

Acknowledgements

This work was supported by JSPS KAKENHI Grant Number JP24H00709.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Aho, A. V. and Ullman, J. D. Optimization of straight line programs. *SIAM Journal on Computing*, 1(1):1–19, 1972.
- Apostolico, A., Atallah, M. J., Larmore, L. L., and McFaddin, S. Efficient parallel algorithms for string editing and related problems. *SIAM Journal on Computing*, 1990.
- Arora, S. and Barak, B. *Computational complexity: a modern approach*. Cambridge University Press, 2009.
- Bae, S., Kim, Y., Bayat, R., Kim, S., Ha, J., Schuster, T., Fisch, A., Harutyunyan, H., Ji, Z., Courville, A., et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*, 2025.
- Barrington, D. A. Bounded-width polynomial-size branching programs recognize exactly those languages in nc. In *ACM symposium on Theory of computing*, 1986.
- Bavandpour, A. A., Huang, X., Rofin, M., and Hahn, M. Lower bounds for chain-of-thought reasoning in hard-attention transformers. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=Oh9sG5ae2b>.
- Buss, S. R. The boolean formula value problem is in ALOG-TIME. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pp. 123–131, 1987.
- Cook, S. A. A taxonomy of problems with fast parallel algorithms. *Information and control*, 64(1-3):2–22, 1985.
- Csordás, R., Irie, K., Schmidhuber, J., Potts, C., and Manning, C. D. MoEUT: Mixture-of-experts universal transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=ZxVrkm7Bjl>.
- de Luca, A. B. and Fountoulakis, K. Simulation of graph algorithms with looped transformers. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=aA2326y3hf>.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- Deng, Y., Choi, Y., and Shieber, S. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*, 2024.
- Erdos, P. and Renyi, A. On random graphs i. *Publ. math. debrecen*, 6(290-297):18, 1959.
- Feng, G., Zhang, B., Gu, Y., Ye, H., He, D., and Wang, L. Towards revealing the mystery behind chain of thought: a theoretical perspective. *Advances in Neural Information Processing Systems*, 36:70757–70798, 2023.
- Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pp. 11398–11442. PMLR, 2023.
- Gibbons, A. and Rytter, W. *Efficient parallel algorithms*. Cambridge University Press, 1989.
- Gozeten, H. A., Ildiz, M. E., Zhang, X., Harutyunyan, H., Rawat, A. S., and Oymak, S. Continuous chain of thought enables parallel exploration and reasoning. *arXiv preprint arXiv:2505.23648*, 2025.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J. E., and Tian, Y. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Itxz7S4Ip3>.
- Jerrum, M. A very simple algorithm for estimating the number of k-colorings of a low-degree graph. *Random Structures & Algorithms*, 7(2):157–165, 1995.
- Jerrum, M. R., Valiant, L. G., and Vazirani, V. V. Random generation of combinatorial structures from a uniform distribution. *Theoretical computer science*, 43:169–188, 1986.
- Karp, R. M. and Luby, M. Monte-carlo algorithms for enumeration and reliability problems. In *24th Annual Symposium on Foundations of Computer Science*, pp. 56–64, 1983.
- Karp, R. M., Luby, M., and Madras, N. Monte-carlo approximation algorithms for enumeration problems. *Journal of algorithms*, 10(3):429–448, 1989.
- Li, Z., Liu, H., Zhou, D., and Ma, T. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning*

- Representations, 2024. URL <https://openreview.net/forum?id=3EWTEy9MTM>.
- Liang, Y., Sha, Z., Shi, Z., Song, Z., and Zhou, Y. Looped relu mlps may be all you need as practical programmable computers. *arXiv preprint arXiv:2410.09375*, 2024.
- Merrill, W. and Sabharwal, A. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- Merrill, W. and Sabharwal, A. The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NjNGLPh8Wh>.
- Merrill, W. and Sabharwal, A. A little depth goes a long way: The expressive power of log-depth transformers. *arXiv preprint arXiv:2503.03961*, 2025a.
- Merrill, W. and Sabharwal, A. Exact expressive power of transformers with padding. *arXiv preprint arXiv:2505.18948*, 2025b.
- Merrill, W., Sabharwal, A., and Smith, N. A. Saturated transformers are constant-depth threshold circuits. *Transactions of the Association for Computational Linguistics*, 2022.
- Mitzenmacher, M. and Upfal, E. *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press, 2017.
- Nowak, F., Svete, A., Butoi, A., and Cotterell, R. On the representational capacity of neural language models with chain-of-thought reasoning. In *Association for Computational Linguistics*, 2024. URL <https://aclanthology.org/2024.acl-long.676/>.
- Sanford, C., Fatemi, B., Hall, E., Tsitsulin, A., Kazemi, M., Halcrow, J., Perozzi, B., and Mirrokni, V. Understanding transformer reasoning capabilities via graph algorithms. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL <https://openreview.net/forum?id=AfzbDw6DSp>.
- Sanford, C., Hsu, D., and Telgarsky, M. Transformers, parallel computation, and logarithmic depth. In *Forty-first International Conference on Machine Learning*, 2024b. URL <https://openreview.net/forum?id=QCZabhKQhB>.
- Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=din0lGfZFd>.
- Schnorr, C.-P. Optimal algorithms for self-reducible problems. In *Proceedings of the Third International Colloquium on Automata, Languages and Programming*, 1976.
- Stockmeyer, L. and Vishkin, U. Simulation of parallel random access machines by circuits. *SIAM Journal on Computing*, 13(2):409–422, 1984.
- Svete, A. and Sabharwal, A. On the reasoning abilities of masked diffusion language models. *arXiv preprint arXiv:2510.13117*, 2025.
- Valiant, L. G. The complexity of enumeration and reliability problems. *siam Journal on Computing*, 8(3):410–421, 1979.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Xu, K. and Sato, I. On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=H4BuhRezCV>.
- Yang, L., Lee, K., Nowak, R. D., and Papailiopoulos, D. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HHbRxoDTxE>.
- Zhu, H., Hao, S., Hu, Z., Jiao, J., Russell, S., and Tian, Y. Reasoning by superposition: A theoretical perspective on chain of continuous thought. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a. URL <https://openreview.net/forum?id=UdOEZgWJLc>.
- Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025b.

A. Formal Definitions

A.1. Notation

Vectors are written in lowercase bold letters (e.g., $\mathbf{x}$) and matrices in uppercase bold letters (e.g., $\mathbf{W}$). The i -th entry of a vector $\mathbf{x}$ is x_i , the vector from the i -th to the j -th entry is denoted by $\mathbf{x}_{i:j}$, and the (i, j) -th entry of a matrix $\mathbf{W}$ is $\mathbf{W}_{i,j}$. We use the symbol $*$ to denote a “don’t care” value (or block of values). For $n \in \mathbb{N}^+$, let $[n] := \{1, 2, \dots, n\}$. We sometimes write column vectors horizontally, e.g., $\mathbf{x} = (x_1, \dots, x_n)$, for brevity. The Hadamard (element-wise) product is $\odot$. $\mathbf{e}_i \in \{0, 1\}^d$ is the i -th standard basis vector, $\mathbf{1}_d \in \mathbb{R}^d$ (or 1_d) the all-ones vector, and $\mathbf{0}_d \in \mathbb{R}^d$ the zero vector. $\mathbf{I}_d \in \mathbb{R}^{d \times d}$ denotes the $d \times d$ identity matrix, and $\mathbf{0}_{m \times n} \in \mathbb{R}^{m \times n}$ the $m \times n$ zero matrix. The indicator function is $\mathbf{1}[\cdot]$, and $\oplus$ denotes block-diagonal concatenation. Functions on scalars or vectors are written in upright letters (e.g., FFN), while functions on matrices are boldface (e.g., **ATTN**). Boldface is also used when scalar- or vector-level functions are extended to sequence level and applied independently to each token (e.g., **FFN**). Finally, $\text{poly}(n)$ denotes the set of functions growing at most polynomially: $\text{poly}(n) := \{f : \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \in \mathbb{N}, \exists c > 0, \forall n \in \mathbb{N}, f(n) \leq c \cdot n^k\}$.

A.2. Transformer Block

We define the computational components of a Transformer block using the notation of (Merrill & Sabharwal, 2025a). Let $\mathbb{F}_s$ denote the set of s -bit floating-point numbers with truncated arithmetic (Definition B.1).

Definition A.1 (Transformer). A Transformer consists of the following components:

1. A word embedding function $\mathbf{WE} : \mathcal{V} \rightarrow \mathbb{F}_s^m$, where $\mathcal{V}$ denotes the vocabulary set.
2. A positional embedding function $\mathbf{PE} : \mathbb{N} \rightarrow \mathbb{F}_s^m$.
3. A multi-head self-attention layer $\mathbf{SA} : \mathbb{F}_s^{m \times N} \rightarrow \mathbb{F}_s^{m \times N}$ for arbitrary sequence length N , parameterized by a matrix $\mathbf{O} : \mathbb{F}_s^{s \times H} \rightarrow \mathbb{F}_s^m$ and, for each head $h \in [H]$ with head size s , matrices $\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h : \mathbb{F}_s^m \rightarrow \mathbb{F}_s^s$. Given an input $\mathbf{x}_i \in \mathbb{F}_s^m$ for each position $i \in [N]$, it computes the query $\mathbf{q}_{i,h} = \mathbf{Q}_h \mathbf{x}_i$, key $\mathbf{k}_{i,h} = \mathbf{K}_h \mathbf{x}_i$, and value $\mathbf{v}_{i,h} = \mathbf{V}_h \mathbf{x}_i$, and outputs $\mathbf{O} \cdot (\mathbf{a}_{i,1}, \dots, \mathbf{a}_{i,H})$, where each attention output $\mathbf{a}_{i,h}$ is defined, for softmax function, as:

$$\mathbf{a}_{i,h} = \sum_{j=1}^{c(i)} \frac{\exp(\mathbf{q}_{i,h}^\top \mathbf{k}_{j,h})}{Z_{i,h}} \cdot \mathbf{v}_{j,h}, \quad Z_{i,h} = \sum_{j=1}^{c(i)} \exp(\mathbf{q}_{i,h}^\top \mathbf{k}_{j,h}), \quad (1)$$

with $c(i) = i$ for causal attention and $c(i) = N$ for full attention. For the saturated hardmax attention (Merrill et al., 2022), each attention output $\mathbf{a}_{i,h}$ is defined as:

$$\mathbf{a}_{i,h} = \sum_{j \in M_{i,h}} \frac{1}{|M_{i,h}|} \mathbf{v}_{j,h}, \quad M_{i,h} = \left\{ j \in [c(i)] \mid \mathbf{q}_{i,h}^\top \mathbf{k}_{j,h} = \max_{j'} \mathbf{q}_{i,h}^\top \mathbf{k}_{j',h} \right\}. \quad (2)$$

4. A feedforward layer $\mathbf{FF} : \mathbb{F}_s^m \rightarrow \mathbb{F}_s^m$ with parameter $\mathbf{W}_1 : \mathbb{F}_s^m \rightarrow \mathbb{F}_s^w$, $\mathbf{W}_2 : \mathbb{F}_s^w \rightarrow \mathbb{F}_s^m$, and $\mathbf{b} \in \mathbb{F}_s^m$, where w is the hidden dimension. Given an input $\mathbf{x}_i \in \mathbb{F}_s^m$, it outputs $\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x}_i + \mathbf{b})$, where $\text{ReLU}(\mathbf{x}) = (\max\{0, x_1\}, \dots, \max\{0, x_m\})^\top$.
5. An output function $\mathbf{OUT} : \mathbb{F}_s^m \rightarrow \mathbb{F}_s^{|\mathcal{V}|}$, parameterized as a linear transformation.

A.3. Chain of Thought

Definition A.2 (CoT). Let the Transformer be defined as the composition:

$$\mathbf{TF}_{\text{dec}} := \mathbf{OUT} \circ (\text{id} + \mathbf{FF}_L) \circ (\text{id} + \mathbf{SA}_L) \circ \dots \circ (\text{id} + \mathbf{FF}_1) \circ (\text{id} + \mathbf{SA}_1) \circ (\mathbf{WE} + \mathbf{PE}), \quad (3)$$

where $\mathbf{SA}_\ell$ and $\mathbf{FF}_\ell$ denote the causal attention and the feedforward layers at depth $\ell \in [L]$, respectively, and id denotes the identity function. The input tokens are first embedded via the word embedding function $\mathbf{WE}$ and the positional encoding $\mathbf{PE}$, and the final output is produced by a linear projection $\mathbf{OUT}$. Given an input sequence $\mathbf{x} = (x_1, \dots, x_n) \in \mathcal{V}^n$, we define the initial sequence as: $f_{\text{cot}}^0(\mathbf{x}) := \mathbf{x}$. Then, the *CoT* computes recursively as:

$$f_{\text{cot}}^{k+1}(\mathbf{x}) := f_{\text{cot}}^k(\mathbf{x}) \cdot \text{Dec}(\mathbf{TF}_{\text{dec}}(f_{\text{cot}}^k(\mathbf{x}))), \quad (4)$$

where $\cdot$ denotes concatenation, and $\text{Dec}(\cdot)$ is a decoding function that maps the output logits to a token in $\mathcal{V}$: in the *deterministic* model, $\text{Dec}(z) := \arg \max_{i \in [|\mathcal{V}|]} z_i$; in the *stochastic* model, $\text{Dec}(z) \sim \text{Multinomial}(z_i / \sum_j z_j)$, assuming $z_i > 0$ for all i . The final output of the CoT model after $T(n)$ steps is defined as the last output length m tokens of $f_{\text{cot}}^{T(n)}(\mathbf{x})$.

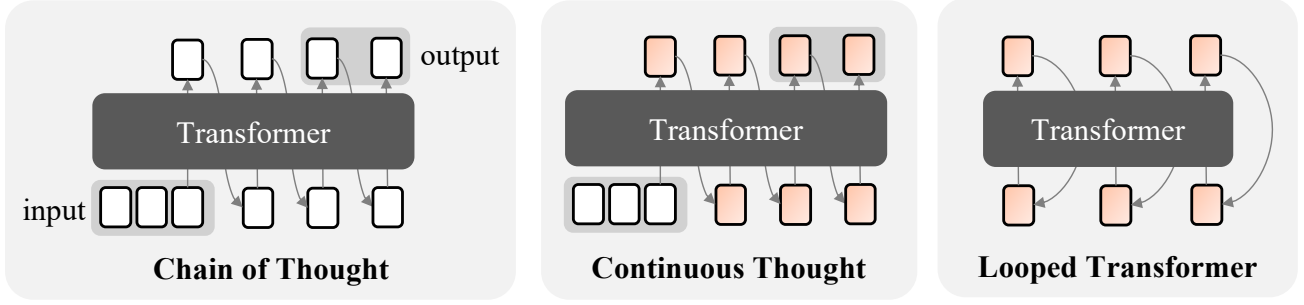


Figure 8. The models of reasoning paradigms based on iterative use of Transformer models.

A.4. Continuous Thought

Definition A.3 (Coconut). Let the Transformer block be defined as the composition:

$$\text{TF}_{\text{dec}} := (\text{id} + \mathbf{FF}_L) \circ (\text{id} + \mathbf{SA}_L) \circ \cdots \circ (\text{id} + \mathbf{FF}_1) \circ (\text{id} + \mathbf{SA}_1), \quad (5)$$

where $\mathbf{SA}_\ell$ and $\mathbf{FF}_\ell$ denote the causal attention and the feedforward layers at depth $\ell \in [L]$, respectively, and id denotes the identity function. Given an input sequence $x = (x_1, \dots, x_n) \in \mathcal{V}^n$, we define the initial sequence as: $f_{\text{cot}}^0(x) := \mathbf{WE}(x) + \mathbf{PE}([n])$, where the input tokens are first embedded via the word embedding function $\mathbf{WE}$ and the positional encoding $\mathbf{PE}$. Then, the *continuous thought* (Coconut) computes recursively as:

$$f_{\text{ct}}^{k+1}(x) := f_{\text{ct}}^k(x) \cdot (\text{TF}_{\text{dec}}(f_{\text{cot}}^k(x) + \mathbf{PE}(k))), \quad (6)$$

where $\cdot$ denotes concatenation. The final output of the model after $T(n)$ steps is defined as the last output length m tokens of $\text{Dec} \left(\mathbf{OUT} \left(f_{\text{ct}}^{T(n)}(x) \right) \right)$, where the final output is produced by a linear projection $\mathbf{OUT}$ and $\text{Dec}(\cdot)$ is a decoding function that maps the output logits to a token in $\mathcal{V}$: in the *deterministic* model, $\text{Dec}(z) := \arg \max_{i \in [|\mathcal{V}|]} z_i$; in the *stochastic* model, $\text{Dec}(z) \sim \text{Multinomial}(z_i / \sum_j z_j)$, assuming $z_i > 0$ for all i .

A.5. Looped Transformer

Definition A.4 (Looped TF). Let the Transformer block be defined as the composition:

$$\text{TF} := (\text{id} + \mathbf{FF}_L) \circ (\text{id} + \mathbf{SA}_L) \circ \cdots \circ (\text{id} + \mathbf{FF}_1) \circ (\text{id} + \mathbf{SA}_1), \quad (7)$$

where $\mathbf{SA}_\ell$ and $\mathbf{FF}_\ell$ denote the (non-causal) self-attention and feedforward layers at depth $\ell \in [L]$.

Given an input token sequence $x = (x_1, \dots, x_n) \in \mathcal{V}^n$, the initial hidden state is: $f_{\text{loop}}^0(x) := \mathbf{WE}(x)$. At each loop iteration k , the hidden state is updated by:

$$f_{\text{loop}}^{k+1}(x) := \text{TF} \left(f_{\text{loop}}^k(x) \right). \quad (8)$$

The final outputs after $T(n)$ loop iterations are decoded as $\text{Dec} \circ \mathbf{OUT} \circ f_{\text{loop}}^{T(n)}(x)$, and the model's prediction is defined as the last output length $m \leq n$ tokens of this projected sequence.

B. Deferred Proofs for Section 3

B.1. Precision Modeling

We focus on signed floating-point numbers, following (Li et al., 2024), but omit exponents for simplicity.

Definition B.1 (Floating-point Representation, cf. (Li et al., 2024)). Consider floating-point numbers with a mantissa part of s bits and a sign bit of 1, totaling $(s + 1)$ bits. We denote the set of such floating-point numbers by $\mathbb{F}_s$, and define $B_s \triangleq \max \mathbb{F}_s$.

Definition B.2 (Correct Rounding, cf. (Li et al., 2024)). For any $x \in \mathbb{R}$ and any closed subset $\mathbb{F} \subset \mathbb{R}$ containing 0, we define the *correct rounding* $\text{round}(x, \mathbb{F})$ as the number in $\mathbb{F}$ closest to x . In particular, rounding to a floating-point number with mantissa part s bits is denoted by $[\cdot]_s$. Rounding applied to vectors is to be operated coordinate-wise.

They also define primitive arithmetic under finite precision by applying rounding after each basic operation. In particular, for multi-operand operations, rounding is applied after each binary operation. Finite-precision summation over more than two numbers is thus defined as follows.

Definition B.3 (Summation with Iterative Rounding (Li et al., 2024)). For any $s, n \in \mathbb{N}^+$ and vector $\mathbf{x} \in \mathbb{R}^n$, define the *summation with iterative rounding to s -bit precision*

$$\text{sum}_s : \bigcup_{n \in \mathbb{N}^+} (\mathbb{F}_s)^n \rightarrow \mathbb{F}_s, \quad (9)$$

where, for any $n \in \mathbb{N}^+$ and $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$,

$$\text{sum}_s(\mathbf{x}) := \left[\left[\cdots \left[[x_1 + x_2]_s + x_3 \right]_s + \cdots + x_{n-1} \right]_s + x_n \right]_s. \quad (10)$$

Based on this definition, all computations in the Transformer block of Definition A.1 can be represented in finite precision. The inner product and the matrix product are defined as

$$\mathbf{x}^\top \mathbf{y} := \text{sum}_s(\mathbf{x} \odot \mathbf{y}), \quad (\mathbf{AB})_{i,j} := \mathbf{A}_{i,:}^\top \mathbf{B}_{:,j}. \quad (11)$$

Throughout this section, we interpret all operations as finite-precision computations as defined above.

B.2. Definition of Assumption 3.4

Our definition of polynomially efficient approximation follows that of Feng et al. (2023), but differs in scope: while their framework targets real-valued functions, ours applies to symbolic functions.

Definition B.4 (Polynomially-efficient approximation). We say that a function $f_n : \Sigma^{\ell(n)} \rightarrow \Sigma$ admits a *polynomially efficient approximation* if, for a sufficiently small error tolerance $0 < \delta \leq \frac{1}{3}$, there exists a feedforward network $\text{FF} : \mathbb{F}_{s(n)}^{\ell(n) \cdot |\Sigma|} \rightarrow \mathbb{F}_{s(n)}^{|\Sigma|}$, $s(n) = O(\log n)$, such that the following holds: for every input $\mathbf{x} = (x_1, \dots, x_{\ell(n)}) \in \Sigma^{\ell(n)}$,

$$\text{FF}(e(x_1), \dots, e(x_{\ell(n)}))_i = \begin{cases} \geq 1 - \delta & \text{if } e(f_n(\mathbf{x})) = e_i, \\ \leq \delta & \text{else,} \end{cases} \quad (12)$$

where $e : \Sigma \rightarrow \{0, 1\}^{|\Sigma|}$ denote the one-hot encoding. Moreover, the number of parameters of the feedforward network is bounded by a polynomial in $\ell(n)$ and $1/\delta$.

B.3. Technical lemmas

In this section, we provide the key components for our constructive proofs.

B.3.1. ORTHOGONAL VECTORS

We follow the notation of (Li et al., 2024). For any positive integer $s \in \mathbb{N}^+$ and $x \in \{0, 1, \dots, 2^s - 1\}$, we denote by $\text{bin}_s(x) \in \{0, 1\}^s$ the standard binary representation of x using s bits, defined such that $x = \sum_{i=1}^s 2^i \cdot (\text{bin}_s(x))_i$. We further define the signed binary encoding of x , denoted by $\text{sbin}_s(x) \in \{-1, 1\}^s$, as $\text{sbin}_s(x) = 2 \cdot \text{bin}_s(x) - (1, \dots, 1)$. Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^s$ be two vectors of the same length. We define their interleaving, denoted by $\mathbf{x} \frown \mathbf{y} \in \mathbb{R}^{2s}$, as follows: $(\mathbf{x} \frown \mathbf{y})_{2i-1} = x_i, (\mathbf{x} \frown \mathbf{y})_{2i} = y_i$ for all $i \in [s]$. The orthogonal vectors under finite-precision arithmetic can be:

Lemma B.5 (Li et al., 2024). For any $s \in \mathbb{N}^+$, let $\mathbf{q}_i = \text{sbin}_s(i) \frown 1_s$ and $\mathbf{k}_i = 2^{s+1} \cdot (\text{sbin}_s(i) \frown (-1_s))$ for all $i \in [2^s - 1]$, it holds that $\langle \mathbf{q}_i, \mathbf{k}_j \rangle_s = -B_s$ if $i \neq j$ and $\langle \mathbf{q}_i, \mathbf{k}_j \rangle_s = 0$ if $i = j$. Since $[\exp(-B_s)]_s \leq [2^{-s-1}]_s = 0$, it follows that $[\exp(\langle \mathbf{q}_i, \mathbf{k}_j \rangle_s)]_s = \mathbf{1}[i = j]$ for all $i, j \in [2^s - 1]$.

B.3.2. POSITION SELECTOR

Lemma B.6. For any $m \in \mathbb{N}^+$ and $\mathbf{x} \in \mathbb{F}_s^m$ with $x_i > 0$ for all $i \in [m]$, there exists a feedforward layer $\text{FF} : \mathbb{F}_s^{2m} \rightarrow \mathbb{F}_s^{2m}$, for any $i \in [m]$, such that

$$(\text{id} + \text{FF})(\mathbf{x}, \mathbf{e}_i) = (\mathbf{x} \odot \mathbf{e}_i, \mathbf{e}_i). \quad (13)$$

Proof. Let the input be $\mathbf{z} = (\mathbf{x}, \mathbf{e}_i) \in \mathbb{F}_s^{2m}$. Set the weight $\mathbf{W}_1 \in \mathbb{F}_s^{m \times 2m}$ and bias $\mathbf{b} \in \mathbb{F}_s^m$ by

$$\mathbf{W}_1 = [\mathbf{I}_m \quad -B_s \mathbf{I}_m], \quad \mathbf{b} = \mathbf{0}, \quad (14)$$

to have $\mathbf{W}_1 \mathbf{z} + \mathbf{b} = \mathbf{x} - B_s \mathbf{e}_i$. Applying the ReLU activation coordinate-wise gives

$$\text{ReLU}(\mathbf{x} - B_s \mathbf{e}_i)_j = \begin{cases} 0, & j = i, \\ x_j, & j \neq i. \end{cases} \quad (15)$$

Hence,

$$\mathbf{h} := \text{ReLU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}) = \mathbf{x} \odot (\mathbf{1} - \mathbf{e}_i). \quad (16)$$

Next, set the second linear layer $\mathbf{W}_2 \in \mathbb{F}_s^{2m \times m}$ by

$$\mathbf{W}_2 = \begin{bmatrix} -\mathbf{I}_m \\ \mathbf{0}_{m \times m} \end{bmatrix}. \quad (17)$$

Thus we have

$$\mathbf{z} + \mathbf{W}_2 \mathbf{h} = \begin{bmatrix} \mathbf{x} \\ \mathbf{e}_i \end{bmatrix} + \begin{bmatrix} -\mathbf{h} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{e}_i \end{bmatrix} + \begin{bmatrix} -\mathbf{x} \odot (\mathbf{1} - \mathbf{e}_i) \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{x} \odot \mathbf{e}_i \\ \mathbf{e}_i \end{bmatrix}. \quad (18)$$

□

B.3.3. FEEDFORWARD LAYERS

Lemma B.7. *Let $N \in \mathbb{N}^+$. For each $i \in [N]$, let $f_i : \Sigma^{\ell_i} \rightarrow \Sigma$ admit polynomially-efficient approximation by FF_i . Then there exists a feedforward layer $\text{FF} : \mathbb{F}_s^{\sum_{i=1}^N \ell_i |\Sigma|} \rightarrow \mathbb{F}_s^{N|\Sigma|}$ such that for every input tuple $\mathbf{x}_i = (x_1^{(i)}, \dots, x_{\ell_i}^{(i)}) \in \Sigma^{\ell_i}$,*

$$\text{FF}\left(\parallel_{i=1}^N (\mathbf{e}(x_1^{(i)}), \dots, \mathbf{e}(x_{\ell_i}^{(i)}))\right) = \parallel_{i=1}^N \text{FF}_i(\mathbf{x}_i), \quad (19)$$

where $\parallel$ denotes concatenation.

Proof. For each $i \in [N]$, by Definition B.4, there exist width $w_i \in \mathbb{N}$ and parameters

$$\mathbf{W}_1^{(i)} \in \mathbb{F}_s^{w_i \times \ell_i |\Sigma|}, \quad \mathbf{W}_2^{(i)} \in \mathbb{F}_s^{|\Sigma| \times w_i}, \quad \mathbf{b}^{(i)} \in \mathbb{F}_s^{w_i} \quad (20)$$

such that $\text{FF}_i(\mathbf{x}_i) = \mathbf{W}_2^{(i)} \text{ReLU}(\mathbf{W}_1^{(i)} \mathbf{e}(\mathbf{x}_i) + \mathbf{b}^{(i)})$.

Now, define block-diagonal matrices

$$\mathbf{W}_1 := \bigoplus_{i=1}^N \mathbf{W}_1^{(i)}, \quad \mathbf{W}_2 := \bigoplus_{i=1}^N \mathbf{W}_2^{(i)}, \quad \mathbf{b} := \bigoplus_{i=1}^N \mathbf{b}^{(i)}. \quad (21)$$

Then the single feedforward layer

$$\text{FF}(\mathbf{x}) := \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}) \quad (22)$$

applies each block independently to its corresponding input, yielding exactly $\text{FF}(\mathbf{x}) = \parallel_{i=1}^N \text{FF}_i(\mathbf{x}_i)$. □

Lemma B.8. *Let $f : \Sigma^\ell \rightarrow \Sigma$ be a function that admits polynomially-efficient approximation (Definition B.4). Then, there exist two feedforward layers*

$$\text{FF}_1 : \mathbb{F}_s^{(1+\ell)|\Sigma|+1} \rightarrow \mathbb{F}_s^{(1+\ell)|\Sigma|+1}, \quad \text{FF}_2 : \mathbb{F}_s^{(1+\ell)|\Sigma|+1} \rightarrow \mathbb{F}_s^{(1+\ell)|\Sigma|+1}, \quad (23)$$

such that, for every $\mathbf{x} = (x_1, \dots, x_\ell) \in \Sigma^\ell$ and $t \in \{0, 1\}$,

$$(\text{id} + \text{FF}_2) \circ (\text{id} + \text{FF}_1)(\mathbf{0}, \mathbf{e}(x_1), \dots, \mathbf{e}(x_\ell), t) = (t \cdot \mathbf{e}(f(\mathbf{x})), \mathbf{e}(x_1), \dots, \mathbf{e}(x_\ell), t). \quad (24)$$

Proof. Let $n := |\Sigma|$ and $L := \ell n$. Write $\mathbf{u} = (e(x_1), \dots, e(x_\ell)) \in \{0, 1\}^L$. By Definition B.4, there exist $w_f \in \mathbb{N}$, matrices $\mathbf{W}_1^{(f)} \in \mathbb{F}_s^{w_f \times L}$, $\mathbf{W}_2^{(f)} \in \mathbb{F}_s^{n \times w_f}$, and bias $\mathbf{b}^{(f)} \in \mathbb{F}_s^{w_f}$ such that

$$\text{FF}_f(\mathbf{u}) := \mathbf{W}_2^{(f)} \text{ReLU}(\mathbf{W}_1^{(f)} \mathbf{u} + \mathbf{b}^{(f)}), \quad \text{FF}_f(\mathbf{u})_i = \begin{cases} \geq 1 - \delta & \text{if } e(f(\mathbf{x})) = e_i, \\ \leq \delta & \text{otherwise.} \end{cases} \quad (25)$$

Set the first layer as

$$\mathbf{W}_1^{(1)} = \begin{bmatrix} \mathbf{0} & \mathbf{W}_1^{(f)} & \mathbf{0}_{w_f \times 1} \\ \mathbf{0} & \mathbf{I}_L & \mathbf{0}_{L \times 1} \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} \mathbf{b}^{(f)} \\ \mathbf{0}_L \end{bmatrix}, \quad \mathbf{W}_2^{(1)} = \begin{bmatrix} \mathbf{W}_2^{(f)} & \mathbf{0} \\ \mathbf{0}_{L \times w_f} & \mathbf{0} \\ \mathbf{0}_{1 \times w_f} & \mathbf{0} \end{bmatrix}, \quad (26)$$

and define $\text{FF}_1(\mathbf{0}, \mathbf{u}, t) := \mathbf{W}_2^{(1)} \text{ReLU}(\mathbf{W}_1^{(1)}(\mathbf{0}, \mathbf{u}, t) + \mathbf{b}^{(1)})$. Then, it holds that

$$\text{FF}_1(\mathbf{0}, \mathbf{u}, t) = \mathbf{W}_2^{(1)} \text{ReLU} \left(\begin{bmatrix} \mathbf{W}_1^{(f)} \mathbf{u} + \mathbf{b}^{(f)} \\ \mathbf{u} \end{bmatrix} \right) \quad (27)$$

$$= \begin{bmatrix} \mathbf{W}_2^{(f)} \text{ReLU}(\mathbf{W}_1^{(f)} \mathbf{u} + \mathbf{b}^{(f)}) \\ \mathbf{0} \end{bmatrix} \quad (28)$$

$$= \begin{bmatrix} \text{FF}_f(\mathbf{u}) \\ \mathbf{0} \end{bmatrix}. \quad (29)$$

Thus we have $(\text{id} + \text{FF}_1)(\mathbf{0}, \mathbf{u}, t) = (\text{FF}_f(\mathbf{u}), \mathbf{u}, t)$.

For the second layer, choose $\delta \leq \frac{1}{3}$ and $M \geq 1$ and set

$$\mathbf{W}_1^{(2)} = \begin{bmatrix} 2\mathbf{I}_n & \mathbf{0}_{n \times L} & M\mathbf{1}_n \\ 2\mathbf{I}_n & \mathbf{0}_{n \times L} & M\mathbf{1}_n \\ \mathbf{I}_n & \mathbf{0}_{n \times L} & \mathbf{0}_{n \times 1} \\ -\mathbf{I}_n & \mathbf{0}_{n \times L} & \mathbf{0}_{n \times 1} \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} -M\mathbf{1}_n \\ (1-M)\mathbf{1}_n \\ \mathbf{0}_n \\ \mathbf{0}_n \end{bmatrix}, \quad \mathbf{W}_2^{(2)} = \begin{bmatrix} \mathbf{I}_n & -\mathbf{I}_n & -\mathbf{I}_n & \mathbf{I}_n \\ \mathbf{0}_{L \times n} & \mathbf{0}_{L \times n} & \mathbf{0}_{L \times n} & \mathbf{0}_{L \times n} \\ \mathbf{0}_{1 \times n} & \mathbf{0}_{1 \times n} & \mathbf{0}_{1 \times n} & \mathbf{0}_{1 \times n} \end{bmatrix}. \quad (30)$$

and define $\text{FF}_2(\mathbf{y}) := \mathbf{W}_2^{(2)} \text{ReLU}(\mathbf{W}_1^{(2)} \mathbf{y} + \mathbf{b}^{(2)})$. Then it holds that, for $\mathbf{z} := \text{FF}_f(\mathbf{u})$,

$$\text{FF}_2(\mathbf{z}, \mathbf{u}, t) = \mathbf{W}_2^{(2)} \begin{bmatrix} \text{ReLU}(2\mathbf{z} + Mt\mathbf{1}_n - M\mathbf{1}_n) \\ \text{ReLU}(2\mathbf{z} + Mt\mathbf{1}_n + (1-M)\mathbf{1}_n) \\ \text{ReLU}(\mathbf{z}) \\ \text{ReLU}(-\mathbf{z}) \end{bmatrix} \quad (31)$$

$$= \mathbf{W}_2^{(2)} \begin{bmatrix} \text{ReLU}(2\mathbf{z} + M(t-1)\mathbf{1}_n) \\ \text{ReLU}(2\mathbf{z} + 1 + M(t-1)\mathbf{1}_n) \\ \mathbf{z} \\ \mathbf{0} \end{bmatrix} \quad (32)$$

$$= \begin{bmatrix} \text{ReLU}(\mathbf{z} - \delta + M(t-1)) - \text{ReLU}(\mathbf{z} - 1 + M(t-1)) - \mathbf{z} \\ \mathbf{0} \\ 0 \end{bmatrix}, \quad (33)$$

where it satisfies that

$$\text{ReLU}(\mathbf{z} - \delta + M(t-1)) - \text{ReLU}(\mathbf{z} - \delta - 1 + M(t-1)) = \begin{cases} e(f(\mathbf{x})) & \text{if } t = 1, \\ \mathbf{0} & \text{if } t = 0. \end{cases} \quad (34)$$

Therefore, the composition satisfies $(\text{id} + \text{FF}_2) \circ (\text{id} + \text{FF}_1)(\mathbf{0}, \mathbf{u}, t) = (t \cdot e(f(\mathbf{x})), \mathbf{u}, t)$. $\square$

B.4. Proof for Theorem 3.5

Proof. Let $G_n = (V_n, E_n)$ be a computation graph, where $\mathcal{F} = \{f_1, f_2, \dots, f_{|\mathcal{F}|}\}$. Each node $v \in V_n$ is labeled by a one-hot vector $e(v) \in \{0, 1\}^{|\mathcal{F}|}$ indicating the function assigned to v from the finite set $\mathcal{F}$. Let $v_1, v_2, \dots, v_{|V_n|}$ denote a fixed topological ordering of V_n , with inputs appearing first and outputs last. For each function $f_i \in \mathcal{F}$, let $C_{f_i}(n) := \max\{|\text{pred}(v)| : v \in V_n, e(v) = e_i\}$. and define $C_{\text{sum}}(n) := \sum_{f \in \mathcal{F}} C_f(n)$, $C_{\text{max}}(n) := \max_{f \in \mathcal{F}} C_f(n)$.

Let the precision be $s(n) = C \cdot \lceil \log_2 n \rceil$ where $C \in \mathbb{N}$ is a sufficiently large integer such that $2^{s(n)} \geq n^C$ exceeds the maximum polynomial step bound under consideration. We denote by $\text{pred}(v_i) \in \mathbb{F}_{s(n)}^{C_{\text{max}}(n)}$ the vector of predecessor indices of node v_i ; that is, if v_i has $d \leq C_{\text{max}}(n)$ incoming edges from nodes $v_{j_1}, \dots, v_{j_d}$, then $\text{pred}(v_i) = (j_1, \dots, j_d, \mathbf{0})$, where zeros are used for padding so that the length is exactly $C_{\text{max}}(n)$.

Let the vocabulary be $\mathcal{V} = \Sigma$. At decoding step k , the model has access to the concatenated sequence

$$(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_k) \in \Sigma^{n+k}, \quad (35)$$

where $x = (x_1, \dots, x_n) \in \Sigma^n$ denotes the input, and y_i is the token generated at the i -th CoT step. For each node v_j , let $v_j(x)$ denote its value on input x . We assume that every intermediate output satisfies $y_i = v_{n+i}(x)$. Under this assumption, we prove by induction that the model generates the next token correctly, i.e., $y_{k+1} = v_{n+k+1}(x)$.

Embedding The embedding at position $i \in [n+k]$, denoted by $h_i^{(0)} \in \mathbb{F}_{s(n)}^m$, where $m := |\Sigma| + |\mathcal{F}| + (1 + C_{\text{max}}(n))s(n) + |\Sigma|C_{\text{sum}}(n)$ is defined as

$$h_i^{(0)} = \left(e(v_i(x)), e(v_{i+1}), \text{sbin}_{s(n)}(i), \text{sbinpred}_{s(n)}(v_{i+1}), \mathbf{0}_{|\Sigma|C_{\text{sum}}(n)} \right), \quad (36)$$

where $e: \Sigma \rightarrow \{0, 1\}^{|\Sigma|}$ denote the one-hot encoding of the symbol and, $\text{sbinpred}_{s(n)}(v) \in \mathbb{F}_{s(n)}^{C_{\text{max}}(n) \cdot s(n)}$ encodes the binary representations of the predecessor indices:

$$\text{sbinpred}_{s(n)}(v_i) := (\text{sbin}_{s(n)}(\text{pred}(v_i)_0), \dots, \text{sbin}_{s(n)}(\text{pred}(v_i)_{C_{\text{max}}(n)})). \quad (37)$$

This embedding is constructed, for $z \in \Sigma$, as

$$\mathbf{WE}(z) = (e(z), \mathbf{0}), \quad \mathbf{PE}(i) = \left(\mathbf{0}, e(v_{i+1}), \text{sbin}_{s(n)}(i), \text{sbinpred}_{s(n)}(v_{i+1}), \mathbf{0} \right). \quad (38)$$

Attention layer The first attention layer consists of $C_{\text{max}}(n)$ heads. The h -th head is configured to attend to the position corresponding to the h -th predecessor. Specifically, for each position i and head $h \in [C_{\text{max}}(n)]$, the attention vectors are defined as:

$$q_{i,h} = \text{sbin}_{s(n)}(\text{pred}(v_{i+1})_h) \frown 1_{s(n)}, \quad (39)$$

$$k_{i,h} = 2^{s(n)+1} \cdot \text{sbin}_{s(n)}(i) \frown (-1_{s(n)}), \quad (40)$$

$$v_{i,h} = e(v_i(x)), \quad (41)$$

where vectors of different lengths are zero-padded to match the dimension. By Lemma B.5, each attention head of the last position $i = n+k$ retrieves the predecessor's value of v_{n+k}

$$a_{n+k,h} = e(v_{\text{pred}(v_{n+k+1})_h}(x)) \quad (42)$$

With an appropriate output projection $\mathbf{O}$ such that

$$\mathbf{O}(a_{i,1}, \dots, a_{i,H}) = (\mathbf{0}, e(v_{\text{pred}(v_{n+k+1})_0}(x)), \dots, e(v_{\text{pred}(v_{n+k+1})_{C_{\text{max}}(n)}}(x)), \mathbf{0}), \quad (43)$$

the hidden state at position $n+k$ after the attention layer is given by

$$h_{n+k}^{(0.5)} = \left(e(v_{n+k}(x)), e(v_{n+k+1}), \text{sbin}_{s(n)}(n+k), \text{sbinpred}_{s(n)}(v_{n+k+1}), \right. \quad (44)$$

$$\left. \underbrace{e(v_{\text{pred}(v_{n+k+1})_0}(x)), \dots, e(v_{\text{pred}(v_{n+k+1})_{C_{\text{max}}(n)}}(x))}_{\text{updated}}, \mathbf{0}_{C_{\text{sum}}(n)|\Sigma|} \right). \quad (45)$$

The second and third attention layers are disabled (i.e., all attention weights are set to zero).

Feed-forward layer By Lemma B.7, a single feed-forward layer can approximate multiple functions by partitioning the input into blocks. The first feed-forward layer then places the arguments, gathered by attention, into the correct positions. By Lemma B.6, where the vector e_i therein corresponds to $1_{|\Sigma|}$ here, the hidden state at the last position, denoted by $h_{n+k}^{(1)}$, becomes

$$\left((h_{n+k}^{(0.5)})_{1:r}, \left\|_{j=1}^{|\mathcal{F}|} (e(v_{\text{pred}(v_{n+k+1})_1}(x)) \cdot 1, \dots, e(v_{\text{pred}(v_{n+k+1})_{C_j(n)}}(x)) \cdot 1) \right\| \right), \quad (46)$$

where $r = |\Sigma| + |\mathcal{F}| + (1 + C_{\max})(n)s(n)$.

By Assumption 3.4 and Lemmas B.7 and B.8, there exist feed-forward layers $\text{FF}_2, \text{FF}_3 : \mathbb{F}_{s(n)}^m \rightarrow \mathbb{F}_{s(n)}^m$ such that, for every input tuple $\mathbf{x}_j := (x_1^{(j)}, \dots, x_{C_j(n)}^{(j)}) \in \Sigma^{C_j(n)}$ for $j \in [|\mathcal{F}|]$ and every $\mathbf{t} \in \{0, 1\}^{|\mathcal{F}|}$, the composition $\mathcal{FF} := (\text{id} + \text{FF}_3) \circ (\text{id} + \text{FF}_2)$ satisfies

$$\mathcal{FF} \left(*, \mathbf{t}, *, \left\|_{j=1}^{|\mathcal{F}|} (e(x_1^{(j)}), \dots, e(x_{C_j(n)}^{(j)})) \right\| \right) = \left(\sum_j^{|\mathcal{F}|} \mathbf{t}_j \cdot e(f_j(\mathbf{x}_j)), * \right), \quad (47)$$

where $*$ denotes an unspecified vector. Since the second and third attention layers are disabled, after the third layer, applying the second and third feed-forward layers FF_2, FF_3 , the hidden state becomes

$$h_{n+k}^{(3)} = \mathcal{FF} \left(h_{n+k}^{(1)} \right) \quad (48)$$

$$= \mathcal{FF} \left(*, e(v_{n+k+1}), *, \left\|_{j=1}^{|\mathcal{F}|} (e(v_{\text{pred}(v_{n+k+1})_1}(x)), \dots, e(v_{\text{pred}(v_{n+k+1})_{C_j(n)}}(x))) \right\| \right) \quad (49)$$

$$= \left(\sum_{j=1}^{|\mathcal{F}|} e(v_{n+k+1})_j \cdot e \left(f_j(e(v_{\text{pred}(v_{n+k+1})_1}(x)), \dots, e(v_{\text{pred}(v_{n+k+1})_{C_j(n)}}(x))) \right), * \right) \quad (50)$$

$$= \left(e \left(f_l(e(v_{\text{pred}(v_{n+k+1})_1}(x)), \dots, e(v_{\text{pred}(v_{n+k+1})_{C_l(n)}}(x))) \right), * \right), \text{ where } e(v_{n+k+1}) = e_l \quad (51)$$

$$= (e(v_{n+k+1}(x)), *). \quad (52)$$

Output layer The final output is given by

$$h_{n+k} = \text{OUT}(h_{n+k}^{(3)}) = [I_{|\Sigma|} \quad 0] h_{n+k}^{(3)} = e(v_{n+k+1}(x)). \quad (53)$$

The decoding function then outputs the symbol corresponding to the maximum score,

$$y_{n+k+1} = \text{Dec}(h_{n+k}) = \arg \max_{j \in [|\Sigma|]} e(v_{n+k+1}(x)) = v_{n+k+1}(x). \quad (54)$$

By induction on k , the model computes the values at all nodes in topological order. The parameter size of the model is determined by the requirements of the feedforward layers, $O(\text{ff_param}(G_n))$. While the dimensions and heads of the attention layers depend on $C_{\max}(n)$, which is precisely what is already required for the feedforward layers to approximate the target functions. $\square$

B.5. Proof of Theorem 3.6 for Looped Transformer

Proof. In the proof, we assume that the computation graph contains at most n output nodes. This assumption is without loss of generality: if the number of output nodes exceeds the number of input nodes, we can simply pad the input with dummy nodes (e.g., fixed zeros), thereby reducing the setting to the same case.

We construct a model in which (1) the attention layer aggregates the inputs, and (2) the looped feed-forward layer performs the computation of all nodes in parallel, as illustrated in Figure 9. We first show that a feedforward layer followed by an attention layer can copy all input tokens to each position. Assume, given an input sequence $x = (x_1, x_2, \dots, x_n) \in \Sigma^n$ and one-hot encoding $e : \Sigma \rightarrow \{0, 1\}^{|\Sigma|}$. Assume each input at position $i \in [n]$ is embedded as

$$h_i = (\mathbf{0}, e(x_i), e_i) \in \{0, 1\}^{n|\Sigma|+|\Sigma|+n}, \quad (55)$$

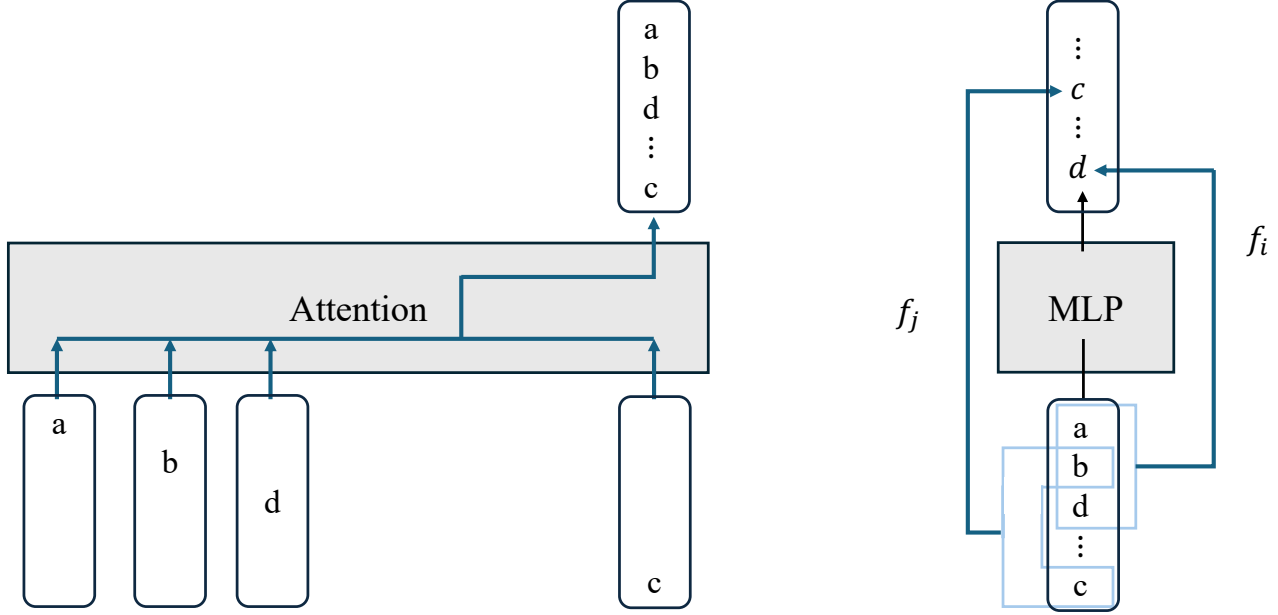


Figure 9. Illustration of the role of the attention and feedforward layers in looped TFs for evaluating DAGs: the attention layer uniformly attends to and aggregates all inputs at each position, while the feedforward layer simultaneously simulates the functions of the nodes.

where $e_i \in \{0, 1\}^n$. By Lemma B.6, when substituting $x = (e(x_1), \dots, e(x_n))$ into the lemma, there exists a feed-forward layer FF_1 such that

$$(\text{id} + \text{FF}_1)(h_i) = ((e_i)_1 \cdot e(x_i), (e_i)_2 \cdot e(x_i), \dots, (e_i)_n \cdot e(x_i), e(x_i), e_i). \quad (56)$$

To aggregate all positions via uniform attention, we use a single-head attention layer with:

$$\mathbf{q}_i = \mathbf{k}_i = \mathbf{1}_{n|\Sigma|}, \quad \mathbf{v}_i = n((e_i)_1 \cdot e(x_i), (e_i)_2 \cdot e(x_i), \dots, (e_i)_n \cdot e(x_i)) \quad \text{for all } i \in [n], \quad (57)$$

with an appropriate output projection, the output of the attention layer, at position i , becomes

$$\frac{1}{n} \sum_{j=1}^n \mathbf{1} \cdot n h_j = \left(\sum_{j=1}^n (e_j)_1 e(x_j), \sum_{j=1}^n (e_j)_2 e(x_j), \dots, \sum_{j=1}^n (e_j)_n e(x_j) \right) \quad (58)$$

$$= (e(x_1), e(x_2), \dots, e(x_n)). \quad (59)$$

Then, we show that the feed-forward layer can encode the entire computation graph into its weights and simulate all nodes simultaneously. Let the flag vector for each node be $(t_1, \dots, t_N) \in \{0, 1\}^N$, where $N := |V_n| = \text{size}(G_n)$. By Lemmas B.7 and B.8, there exist feed-forward layers $\text{FF}_2, \text{FF}_3 : \mathbb{F}_s^{N(|\Sigma|+1)} \rightarrow \mathbb{F}_s^{N(|\Sigma|+1)}$ such that, for the input vector $(z_1, \dots, z_N) \in \Sigma^N$,

$$\mathcal{FF}\left(e(z_1), t_1, \dots, e(z_N), t_N\right) = \left\|_{i=1}^N \left(t_i \cdot e(f_{v_i}(z^{(i)})), \frac{1}{m_i} \sum_{j=1}^{m_i} t_{p_{i,j}} \right), \right. \quad (60)$$

where $\mathcal{FF} := (\text{id} + \text{FF}_3) \circ (\text{id} + \text{FF}_2)$. Here, f_{v_i} denotes the function associated with node v_i , and $p_{i,1}, \dots, p_{i,m_i}$ denote the indices of the predecessor nodes of v_i , and $z^{(i)} = (z_{p_{i,1}}, \dots, z_{p_{i,m_i}})$ denotes their values. The last term $\frac{1}{m_i} \sum_{j=1}^{m_i} t_{p_{i,j}}$ can be obtained using a linear layer.

For the k -th loop, assume by induction that the hidden state is

$$\mathbf{h}(k) := (t_{1,k-1} \cdot e(v_1(x)), t_{1,k}, t_{2,k-1} \cdot e(v_2(x)), t_{2,k}, \dots, t_{N,k-1} \cdot e(v_N(x)), t_{N,k}), \quad (61)$$

where $v_i(x) \in \Sigma$ denotes the value computed by node v_i given the input x , and $t_{i,k} \in \{0, 1\}$ indicates whether node v_i lies within depth at most k . Under this assumption, it holds that

$$\mathcal{FF}(\mathbf{h}(k)) = \left\|_{i=1}^N \left(t_{i,k} \cdot \mathbf{e}(f_{v_i}(v_{p_{i,1}}(x), v_{p_{i,2}}(x), \dots, v_{p_{i,m_i}}(x))), \frac{1}{m_i} \sum_{j=1}^{m_i} t_{(p_{i,j},k)} \right), \right. \quad (62)$$

$$= \left\|_{i=1}^N \left(t_{i,k} \cdot \mathbf{e}(v_i(x)), t_{i,k+1} \right) = \mathbf{h}(k+1). \quad (63)$$

To extract the output node corresponding to each position denoted by o_i , in the final loop iteration, by Lemma B.6, there exists a feedforward layer FF_4 such that

$$(\text{id} + \text{FF}_4)(\mathbf{h}(k), \mathbf{e}_{o_i}) = (t_{o_i,k} \cdot \mathbf{e}(v_{o_i}(x)), *). \quad (64)$$

Summary We construct the looped model as follows. Each input token $x_i \in \Sigma$ at position $i \in [n]$ is embedded as

$$\mathbf{h}_i^{(0)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{e}(x_i), 0, \mathbf{e}(x_i), 0, \dots, \mathbf{e}(x_i), 0, \mathbf{0}_{(1+|\Sigma|)(N-n)+|\Sigma|}) \in \{0, 1\}^{2n+(1+|\Sigma|)N+|\Sigma|}. \quad (65)$$

The first attention layer is an identity map, while the first feedforward layers compute

$$\mathbf{h}_i^{(1)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, (\mathbf{e}_i)_1 \cdot \mathbf{e}(x_i), 0, (\mathbf{e}_i)_2 \cdot \mathbf{e}(x_i), 0, \dots, (\mathbf{e}_i)_n \cdot \mathbf{e}(x_i), 0, \mathbf{0}_{(1+|\Sigma|)(N-n)+|\Sigma|}). \quad (66)$$

The second attention layer uniformly gathers all positions and appends a constant 1 to each block:

$$\mathbf{h}_i^{(1.5)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{e}(x_1), 1, \mathbf{e}(x_2), 1, \dots, \mathbf{e}(x_n), 1, \mathbf{0}_{(1+|\Sigma|)(N-n)+|\Sigma|}) \quad (67)$$

$$= (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{h}(0), \mathbf{0}_{|\Sigma|}). \quad (68)$$

We now proceed by induction. Assume that at the k -th iteration, the output after the second attention layer at position i is

$$\mathbf{h}_{k,i}^{(1.5)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{h}(k), t_{o_i,k-1} \cdot \mathbf{e}(v_{o_i}(x))), \quad (69)$$

where $t_{o_i,k-1} := (\mathbf{h}_{k-1,i}^{(1.5)})_{2n+(1+|\Sigma|)(o_i-1)}$ denotes the indicator flag showing whether the o_i -th node has been reached, i.e., whether it lies at depth $k-1$.

After passing through the second feedforward layer, the third attention layer with all weights set to zero, and the third feedforward layer, the hidden state updates to

$$\mathbf{h}_{k,i}^{(3)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{h}(k+1), t_{o_i,k-1} \cdot \mathbf{e}(v_{o_i}(x))). \quad (70)$$

After the fourth feedforward layer, the hidden state becomes

$$\mathbf{h}_{k,i}^{(4)} = (\mathbf{e}_i, \mathbf{e}_{o_i}, \mathbf{h}(k+1), t_{o_i,k} \cdot \mathbf{e}(v_{o_i}(x))). \quad (71)$$

By induction, after the final iteration of depth $\text{depth}(G_n)$ of the computation graph G_n , we obtain

$$\mathbf{h}_{\text{depth}(G_n),i}^{(4)} = (*, t_{o_i,\text{depth}(G_n)} \cdot \mathbf{e}(v_{o_i}(x))) = (*, \mathbf{e}(v_{o_i}(x))). \quad (72)$$

The final output is given by

$$z_i = \text{OUT}(\mathbf{h}_{\text{depth}(G_n),i}^{(4)}) = [\mathbf{0} \quad \mathbf{I}_{|\Sigma|}] (\mathbf{h}_{\text{depth}(G_n),i}^{(4)}) = \mathbf{e}(v_{o_i}(x)). \quad (73)$$

The decoding function then selects the symbol corresponding to the maximum score:

$$y_i = \text{Dec}(z_i) = \arg \max_{j \in [|\Sigma|]} (\mathbf{e}(v_{o_i}(x)))_j = v_{o_i}(x). \quad (74)$$

The parameter size grows proportionally with the input dimension size(G_n), since the function must be approximated along each dimension. Therefore, it can be bounded by $O(\text{ff_param}(G_n) \cdot \text{size}(G_n))$. $\square$

Discussion: Our proof compresses the entire input into a single position before applying an arbitrary feed-forward computation, which may appear to deviate from the standard Transformer architecture. An alternative approach is to distribute computation across multiple positions, as shown by (Sanford et al., 2024b), which can reduce the required embedding dimension. We nevertheless adopt the single-position construction to isolate the core characteristics of looped models: attention is used purely as an information aggregation mechanism, a single Transformer layer suffices, and all computation is carried out by the feed-forward network in latent space. This latent-space computation is strictly more expressive than computation in the language space. This is supported by recent work for looped ReLUs (Liang et al., 2024).

B.6. Proof of Theorem 3.6 for Continuous Thought

Proof. The proofs are based on the construction for CoT and looped TF. Let the vocabulary be $\mathcal{V} = \Sigma$ and each node $v \in V_n$ is labeled by a one-hot vector $e(v) \in \{0, 1\}^{|\mathcal{F}|}$. At decoding step k , the model has access to the concatenated sequence

$$(x_1, x_2, \dots, x_n, h_1, h_2, \dots, h_k) \quad (75)$$

where $x = (x_1, \dots, x_n) \in \Sigma^n$ denotes the input, and $h_i \in \mathbb{F}_s^m$ is the hidden state generated at the i -th Coconut step. For each node v_j , let $v_j(x)$ denote its value on input x . We assume that

$$h_k := (t_{1,k-1} \cdot e(v_1(x)), t_{1,k}, t_{2,k-1} \cdot e(v_2(x)), t_{2,k}, \dots, t_{N,k-1} \cdot e(v_N(x)), t_{N,k}, e'_k, \mathbf{0}_{|\Sigma|}),$$

where $N := \text{size}(G_n)$, $t_{i,k} \in \{0, 1\}$ indicates whether node v_i lies within depth at most k , and $e : \Sigma \rightarrow \{0, 1\}^{|\Sigma|}$ denotes one-hot encoding, and the vector $e'_k \in \{0, 1\}^N$ is defined by

$$e'_k = \begin{cases} \mathbf{0} & k < \text{depth}(G_n), \\ e(v_{o_i}) & k = \text{depth}(G_n) + i, \end{cases} \quad (76)$$

where v_{o_i} denotes the i -th output node, and e'_k can be encoded by positional embeddings. Under this assumption, we prove by induction that the model generates $h_{k+1}(x)$.

We first consider the base case $k = 1$, in which the model receives only x . We focus on the final token. Using the same construction as in the looped TF, we can show that a single feed-forward layer followed by an attention layer suffices to copy all input tokens to every position. Specifically, the hidden representation at the last position becomes

$$h_1 = (e(v_1(x)), 1, \dots, e(v_n(x)), 1, \mathbf{0}, 1, \mathbf{0}, e'_0, \mathbf{0}), \quad (77)$$

where $v_i(x) = x_i$ for all $i \in [n]$, corresponding to the input nodes. In what follows, we consider the case $k > 1$. By the same argument as for Looped TF, based on Equation (62), we show that the feed-forward layers can encode the entire computation graph into their weights and simulate all nodes simultaneously. That is, there exist two feed-forward layers whose composition, denoted by $\mathcal{FF}$, satisfies $\mathcal{FF}(h_k) = h_{k+1}$.

Consider the last feed-forward layer applied after the zero-weight attention layer. By substituting $e_i = e'_k$ and $x = (h_k)_{1:m-|\Sigma|}$ into Lemma B.6, there exists a feed-forward network FF such that

$$(\text{id} + \text{FF})(h_k) = \begin{cases} h_{k+1}, & \text{if } k < \text{depth}(G_n), \\ ((h_{k+1})_{1:m-|\Sigma|}, e(v_{o_i}(x))), & \text{if } k = \text{depth}(G_n) + i. \end{cases} \quad (78)$$

In particular, for all $k < \text{depth}(G_n)$, the output of the last token, for the input $(x_1, x_2, \dots, x_n, h_1, h_2, \dots, h_k)$ is h_{k+1} . For the final positions corresponding to output nodes, namely $k = \text{depth}(G_n) + i$, the hidden state h_k contains the embedding of the output symbol $v_{o_i}(x)$. Applying the output projection yields

$$\text{OUT}(h_k) = [\mathbf{0} \quad \mathbf{I}_{|\Sigma|}] h_k = e(v_{o_i}(x)). \quad (79)$$

Finally, the decoding function outputs the symbol in Σ corresponding to the maximum coordinate of $\text{OUT}(h_k)$. $\square$

B.7. Proof for Theorem 3.12

For the upper bound $\text{Loop}[\log^k n, \text{poly}(n), 1 \text{ (resp. } \log n)] \subseteq \text{AC}^k \text{ (resp. TC}^k)$, we follow the argument of (Li et al., 2024). Their key observation is that a restricted form of automaton can model iterative computation under constant precision: the rounding operation preserves monotonicity, and constant precision yields counter-free, restricted state spaces, which are therefore computable by AC^0 circuits. In the case of polynomial precision, prefix summation can be simulated by TC^0 , which also allows the detection and correction of rounding in floating-point arithmetic.

Theorem B.9 (Li et al., 2024). *For $s(n) \in \text{poly}(n)$, $\text{sum}_{s(n)} : (\mathbb{F}_{s(n)})^n \rightarrow \mathbb{F}_{s(n)}$ is computable by TC^0 circuits, and by AC^0 circuits when $s(n)$ is constant.*

It has also been shown that gates can be efficiently simulated by feedforward layers.

Lemma B.10 (Li et al., 2024). *Unbounded-fanin AND, OR (resp. MAJORITY) : $\{0, 1\}^n \rightarrow \{0, 1\}$ can be simulated by a two-layer feedforward ReLU network with constant (resp. $\log n$) bits of precision constant hidden dimension and additional n constant inputs of value 1.*

Proof for Theorem 3.12. $\text{Loop}[\log^k n, \text{poly}(n), 1 \text{ (resp. } \log n)] \subseteq \text{AC}^k \text{ (resp. TC}^k)$: In Transformers, constant-depth computation is defined by Summation with Iterative Rounding (see Definition B.3), which by Theorem B.9 can be simulated in AC^0 (resp. TC^0). A looped TF simply stacks these computations vertically through iteration, and thus the result follows.

$\text{AC}^k \text{ (resp. TC}^k) \subseteq \text{Loop}[\log^k n, \text{poly}(n), 1 \text{ (resp. } \log n)]$: Since Boolean circuits are DAGs, the claim follows directly from Theorem 3.10 together with Lemma B.10. $\square$

C. Deferred Proofs for Section 4

C.1. Definition for Models of Computation

We model a language model as a probabilistic process that, given an input and a generated prefix, produces either an internal reasoning state or an output token. This process induces a probability distribution over final outputs. We first formalize CoT under this setting. We focus on the saturated hardmax attention as in (Merrill et al., 2022; Nowak et al., 2024).

Definition C.1 (Language model with CoT). Let $\mathcal{V}$ be a vocabulary. Given an input $x \in \mathcal{V}^*$, a language model with CoT stochastically generates a sequence of output blocks of the form

$$(r_1, e, y_1, e', r_2, e, y_2, e', \dots, r_m, e, y_m, e'), \quad (80)$$

where each $r_i \in \mathcal{V}^*$ represents an explicit reasoning trace, $y_i \in \mathcal{V}$ is an output token, and $e, e' \in \mathcal{V}$ are special delimiter tokens. The final output is the string $y_1 \dots y_m$. Generation proceeds autoregressively: at iteration i , the model generates a reasoning segment r_i followed by an output token y_i , conditioned on the input x , the previously generated outputs $y_{<i}$, and prior reasoning segments $r_{<i}$. We denote by $p(y \mid x)$ the resulting distribution over final outputs.

Then we define the language models with latent thought reasoning: Coconut and looped TF.

Definition C.2 (Language model with Coconut). Let $\mathcal{V}$ be a vocabulary. Given an input $x \in \mathcal{V}^*$, a language model with Coconut generates a sequence of blocks

$$(r_1, y_1, r_2, y_2, \dots, r_m, y_m), \quad (81)$$

where each $r_i \in \mathbb{R}^{d^*}$ represents an internal continuous reasoning state, and each $y_i \in \mathcal{V}$ is an output token.

Generation proceeds autoregressively. At each iteration, the internal continuous reasoning state r_i is generated deterministically as a function of the input x , the previously generated outputs $y_{<i}$, and the prior internal states $r_{<i}$, while the output token y_i is generated stochastically. The decision of when to emit an output token is determined internally by the model.

For looped TF models, the computation proceeds through internally repeated iterations before producing any output tokens. The number of iterations is determined by the model as a function of the input.

Definition C.3 (Language model with looped TF). Given an input $x \in \mathcal{V}^*$, a looped TF produces an output sequence autoregressively. At each iteration $i \in [m]$, the model internally performs a number of repeated transformation steps before emitting an output token y_i , conditioned on the input x and the previously generated outputs $y_{<i}$. The number of internal

loop iterations required to generate each output token is determined internally by the model as a function of the input x and the generated prefix $y_{<i}$.

We define complexity classes corresponding to language models under different reasoning paradigms. In contrast to the non-uniform models typically used in parallel computation analysis, we adopt a **uniform setting** analogous to Turing machines: a single model with a fixed set of parameters is applied to all input lengths, while the number of reasoning steps is allowed to grow as a function of the input size n . Following the convention in (Merrill & Sabharwal, 2024), we allow $O(\log n)$ bits of numerical precision to represent positional embeddings and internal activations. Furthermore, we extend the output space beyond simple binary decisions. Specifically, the model’s output is not restricted to Σ^k , but can be an arbitrary finite binary string in Σ^* . This extension enables the model to represent functions of the form $f : \Sigma^* \rightarrow \mathbb{R}$, thereby capturing counting, probabilistic modeling, and approximation tasks.

Definition C.4 (Complexity Classes pCoT, pCT, and pLOOP). Let $\text{pCoT}[T(n)]$, $\text{pCT}[T(n)]$, and $\text{pLOOP}[T(n)]$ denote the classes of probabilistic computations that define an output distribution $p : \Sigma^* \times \Sigma^* \rightarrow [0, 1]$. A function f belongs to such a class if there exists a language model $\mathcal{M}$, employing CoT, Coconut, or looped TF, respectively, such that for any input $x \in \Sigma^n$, the model induces the distribution $p(x, \cdot)$ within $O(T(n))$ steps using $O(\log n)$ bits of numerical precision.

C.2. Lemma: Universality of Chain of Thought

Definition C.5. A probabilistic Turing machine $M = (Q, \Sigma, \Gamma, q_0, q_1, \delta_1, \delta_2)$ is defined as:

- Q is a finite set of states,
- Σ is the input/output alphabet,
- Γ is the tape alphabet, with $\Sigma \subseteq \Gamma$ and a distinguished blank symbol $\sqcup \in \Gamma$,
- $q_0 \in Q$ denotes the initial state, and $q_1 \in Q$ denotes the final state,
- $\delta_1, \delta_2 : Q \times \Gamma \rightarrow Q \times \Gamma \times (\Sigma \cup \{\varepsilon\}) \times \{L, S, R\}$ are two transition functions.

At each step, M chooses uniformly at random between δ_1 and δ_2 . Each transition $\delta_i(q, a) = (q', b, \sigma, D)$ is interpreted as: writing $b \in \Gamma$ on the work tape, writing $\sigma \in \Sigma$ on the output tape, where ε means that nothing is written, and moving the tape head in direction $D \in \{L, S, R\}$, where L = left, R = right, and S = stay. The output of M is defined to be the string remaining on the output tape when the machine halts.

Prior work has shown that probabilistic CoT can simulate any such PTM, thereby demonstrating its universality.

Lemma C.6 (Nowak et al., 2024). *Let M be a PTM with input and output alphabet Σ , and running time bounded by $T(n)$ on inputs of length n . Then there exists a log-precision CoT model with stochastic decoding, denoted CoT_M , such that the induced output distribution of CoT_M coincides exactly with that of M . Formally, for every input string $x \in \Sigma^*$ and output string $y \in \Sigma^*$,*

$$\Pr[\text{CoT}_M(x) = y] = \Pr[M(x) = y]. \quad (82)$$

Moreover, the number of reasoning steps of CoT_M is bounded by $\text{poly}(|x|)$.

C.3. Self-Reducibility and Complexity of Approximate Counting

Here, we provide the definitions of relation and associated counting problems.

Definition C.7 (Relation). A *relation* over an alphabet Σ is a subset $R \subseteq \Sigma^* \times \Sigma^*$. For an input $x \in \Sigma^*$, we denote

$$R(x) := \{y \in \Sigma^* : (x, y) \in R\}.$$

Definition C.8 (Counting). Given a relation R , the associated counting function is defined as

$$N_R : \Sigma^* \rightarrow \mathbb{N}, \quad N_R(x) := |R(x)|.$$

Definition C.9 (p -relation). A relation $R \subseteq \Sigma^* \times \Sigma^*$ is called a p -relation if

- the membership $(x, y) \in R$ can be decided in time polynomial in $|x|$, and
- there exists a polynomial p such that for all $(x, y) \in R$, we have $|y| \leq p(|x|)$.

Definition C.10. The *extension counting function* associated with a relation $R \subseteq \Sigma^* \times \Sigma^*$ is

$$\text{EXT}_R : \Sigma^* \times \Sigma^* \rightarrow \mathbb{N}, \quad \text{EXT}_R(x, w) := |\{z \in \Sigma^* : (x, wz) \in R\}|.$$

The complexity class $\#P$ consists of counting problems associated with p -relations (Valiant, 1979). Then, we provide definitions of schemes for approximate counting. In the remainder of this paper, we say that an algorithm produces an output approximating $f(x)$ within ratio $1 + \varepsilon$ if its output $\hat{f}(x)$ satisfies

$$(1 - \varepsilon)f(x) \leq \hat{f}(x) \leq (1 + \varepsilon)f(x),$$

Definition C.11 (FPTAS). An algorithm is called a *fully polynomial-time approximation scheme* (FPTAS) for a function f if, for any input x and any $\varepsilon > 0$, it produces an output approximating $f(x)$ within ratio $1 + \varepsilon$, and runs in time polynomial in $|x|$ and $1/\varepsilon$.

Definition C.12 (FPRAS). An algorithm is a *fully polynomial-time randomized approximation scheme* (FPRAS) for a function f if, for any input x and any $\varepsilon > 0$ and $\delta > 0$, it produces an output approximating $f(x)$ within ratio $1 + \varepsilon$ with probability at least $1 - \delta$, and runs in time polynomial in $|x|$, $1/\varepsilon$, and $\log(1/\delta)$.

The following proposition formalizes the relationship between approximate counting and the extension counting function.

Proposition C.13 (Jerrum et al., 1986). Let R be a p -relation and let $x \in \Sigma^n$. Let $m = p(n)$ and define $r = 1 + \frac{\varepsilon}{2m}$. If there exists a FPRAS that approximates $\text{Ext}_R(x, w)$ within ratio r for all prefixes w with probability at least $1 - \delta/m$, then there exists an FPRAS that approximates $N_R(x)$ within ratio $1 + \varepsilon$ with probability at least $1 - \delta$.

This reduction transforms the global counting problem into a sequence of local estimation steps.

Proposition C.14. For any $0 < \varepsilon \leq 1$ and any integer $m \geq 1$, it holds that

$$\left(1 + \frac{\varepsilon}{2m}\right)^m < 1 + \varepsilon. \quad (83)$$

Proof. We use the standard inequality $1 + x \leq e^x$, which holds for all $x \in \mathbb{R}$. Substituting $x = \frac{\varepsilon}{2m}$, we have:

$$\left(1 + \frac{\varepsilon}{2m}\right)^m \leq \left(\exp\left(\frac{\varepsilon}{2m}\right)\right)^m = e^{\varepsilon/2}. \quad (84)$$

For $0 < \varepsilon \leq 1$, we show that $e^{\varepsilon/2} < 1 + \varepsilon$. Let $f(\varepsilon) = 1 + \varepsilon - e^{\varepsilon/2}$. Then $f(0) = 0$ and $f'(\varepsilon) = 1 - \frac{1}{2}e^{\varepsilon/2}$. Since $e^{\varepsilon/2} \leq e^{1/2} < 2$ for all $\varepsilon \in [0, 1]$, it follows that $f'(\varepsilon) > 0$. Thus, f is strictly increasing on $[0, 1]$, implying $f(\varepsilon) > f(0) = 0$, or equivalently $e^{\varepsilon/2} < 1 + \varepsilon$. $\square$

Proof for Proposition C.13. Let $x \in \Sigma^n$ and $m = p(n)$. We express $N_R(x)$ as a product of m ratios. Let $w = y_1 y_2 \dots y_m$ be a witness, and let $w^{(i)}$ denote its prefix of length i . We can write:

$$N_R(x) = \text{Ext}_R(x, \lambda) = \prod_{i=1}^m \frac{\text{Ext}_R(x, w^{(i-1)})}{\text{Ext}_R(x, w^{(i)})} \cdot \text{Ext}_R(x, w^{(m)}). \quad (85)$$

In the standard self-reducibility framework, we estimate each ratio $\rho_i = \text{Ext}_R(x, w^{(i-1)})/\text{Ext}_R(x, w^{(i)})$ using the assumed approximation scheme. Let A_i be the estimator for the i -th ratio such that:

$$\mathbb{P}\left[\frac{1}{r} \leq \frac{A_i}{\rho_i} \leq r\right] \geq 1 - \frac{\delta}{m}. \quad (86)$$

By the union bound, the probability that at least one of these m estimations fails to stay within the ratio r is at most $m \cdot (\delta/m) = \delta$. Therefore, with probability at least $1 - \delta$, all m estimations are successful. Under this condition, the final estimate $\hat{N} = \prod_{i=1}^m A_i$ satisfies:

$$\frac{1}{r^m} \leq \frac{\hat{N}}{N_R(x)} \leq r^m. \quad (87)$$

From Proposition C.14, we have $r^m = (1 + \frac{\varepsilon}{2m})^m < 1 + \varepsilon$. Furthermore, for $0 < \varepsilon \leq 1$, it holds that $1/r^m > (1 + \varepsilon)^{-1} \geq 1 - \varepsilon$, ensuring a relative error of at most ε .

Regarding complexity, each of the m calls to the FPRAS for Ext_R runs in time $\text{poly}(n, (r-1)^{-1}, \log(m/\delta))$. Substituting $r-1 = \varepsilon/2m$, the runtime per call is $\text{poly}(n, m/\varepsilon, \log(m/\delta))$. Since $m = p(n)$ is a polynomial in n , the total running time is $\text{poly}(n, 1/\varepsilon, \log(1/\delta))$, which satisfies the requirements for an FPRAS. $\square$

We have shown that approximate counting reduces to approximating the *extension counting function*. For a general relation R , however, the connection between the extension counting function and the original counting problem for R remains unclear. This gap is bridged by the notion of *self-reducibility*: for self-reducible relations, the extension counting function can be reduced back to the original counting problem for R .

Definition C.15 (Schnorr, 1976). A relation $R \subseteq \Sigma^* \times \Sigma^*$ is *self-reducible* if:

1. There exists a polynomial-time computable function $g \in \Sigma^* \rightarrow \mathbb{N}$ s.t., $(x, y) \in R \Rightarrow |y| = g(x)$;
2. There exists a polynomial-time Turing machine that decides membership in R .
3. There exist polynomial-time computable functions $\psi \in \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$ and $\sigma \in \Sigma^* \rightarrow \mathbb{N}$ s.t.

$$\sigma(x) = O(\log |x|), \quad (88)$$

$$g(x) > 0 \Rightarrow \sigma(x) > 0 \quad \forall x \in \Sigma^*, \quad (89)$$

$$|\psi(x, w)| \leq |x| \quad \forall x, w \in \Sigma^*, \quad (90)$$

and such that, for all $x \in \Sigma^*$, $y = y_1 \dots y_n \in \Sigma^*$,

$$\langle x, y_1, \dots, y_n \rangle \in R \iff \langle \psi(x, y_1 \dots y_{\sigma(x)}), y_{\sigma(x)+1}, \dots, y_n \rangle \in R. \quad (91)$$

For example, SAT is self-reducible: by fixing a prefix of variables and applying the reduction map ψ , the problem is simplified to a smaller instance whose solutions extend the chosen prefix. Consider the Boolean formula $F = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$, and suppose we fix the first variable to $x_1 = 1$. The residual instance is obtained by applying $\psi(F, (1))$, which substitutes $x_1 = 1$ and simplifies the formula by deleting satisfied clauses and removing falsified literals: $\psi(F, (1)) = (x_3) \wedge (\neg x_2 \vee \neg x_3)$. The unit clause (x_3) forces $x_3 = 1$, which in turn simplifies $(\neg x_2 \vee \neg x_3)$ to $\neg x_2$, yielding $x_2 = 0$. Hence the unique residual assignment is $(x_2, x_3) = (0, 1)$, and together with the prefix $x_1 = 1$, we obtain the satisfying assignment $(x_1, x_2, x_3) = (1, 0, 1)$.

For self-reducible relations, the extension counting function is no harder to approximate than the original counting problem.

Proposition C.16 (Jerrum et al., 1986). *Let R be self-reducible. If there exists an FPRAS for N_R , then there exists an FPRAS for Ext_R .*

Proof. By the definition of self-reducibility, the extension function $\text{Ext}_R(x, w)$, which counts the number of strings y such that $(x, wy) \in R$, can be mapped to the counting problem of a modified instance. Specifically, for any prefix w where $|w| \leq \sigma(x)$, there exists a polynomial-time computable mapping ψ such that:

$$\text{Ext}_R(x, w) = |\{y \in \Sigma^{\sigma(x)-|w|} : (x, wy) \in R\}| = N_R(\psi(x, w)). \quad (92)$$

Since R is self-reducible, the instance $x_w = \psi(x, w)$ can be constructed in polynomial time relative to $|x|$. By the hypothesis, there exists an FPRAS for N_R , which provides a randomized $(1 \pm \epsilon)$ -approximation of $N_R(x_w)$ in time polynomial in $|x_w|$ and $1/\epsilon$. Consequently, this algorithm serves as an FPRAS for $\text{Ext}_R(x, w)$, as it runs in polynomial time and satisfies the required approximation guarantees. $\square$

C.4. Proof for Theorem 4.3

Lemma C.17 (Formal Statement of Lemma 4.3). *Assume that $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations. There exists a self-reducible relation R and an associated function $\text{Ext}_R : \Sigma^* \times \Sigma^* \rightarrow \mathbb{N}$ defined by $\text{Ext}_R(x, y_{<i}) := |\{z \in \Sigma^* : (x, y_{<i}z) \in R\}|$ such that language models with CoT using polynomially many reasoning steps, which output a distribution for a given input $(x, y_{<i}) \in \Sigma^n \times \Sigma^*$ by using a linear head for the last hidden state before emitting the output token, admit an FPRAS for Ext_R , whereas no latent thought with polynomially many iterations admits the same approximation guarantee using a linear head for the last hidden state before emitting the output token.*

Proof. By Lemma C.6, CoT can simulate any probabilistic Turing machine running in polynomial time; thus, it can implement an FPRAS for N_R . By Proposition C.16, the existence of an FPRAS for N_R further implies the existence of an FPRAS for the extension function Ext_R . On the other hand, latent thought consisting of a polynomial number of iterations can always be simulated by a deterministic polynomial-time Turing machine, provided that all state transitions in the latent computation are deterministic. Consequently, if such a latent thought process were to admit an FPRAS for Ext_R , it would effectively yield a deterministic polynomial-time approximation scheme. By Proposition C.16, the existence of such a scheme for Ext_R would imply the existence of an FPTAS for N_R . However, under the standard complexity-theoretic assumption that $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations, there exists a self-reducible relation R whose counting function admits an FPRAS but no FPTAS. This yields a contradiction. $\square$

C.5. Proof for Theorem 4.4

Definition C.18 (FPAUS). Uniform generation asks to sample an element y uniformly at random from $R(x)$. A *fully polynomial almost uniform sampler* (FPAUS) for R is a randomized algorithm that, given an input $x \in \Sigma^*$ and an accuracy parameter $\varepsilon > 0$, runs in time polynomial in $|x|$ and $\log(1/\varepsilon)$, and outputs a distribution $q(\cdot \mid x)$ such that

$$\|q(\cdot \mid x) - U(R(x))\|_{\text{TV}} \leq \varepsilon,$$

where $U(R(x))$ denotes the uniform distribution over the set $R(x)$, and $\|\cdot\|_{\text{TV}}$ denotes total variation distance.

For self-reducible relations, the following holds.

Theorem C.19 (Jerrum et al., 1986). *Let R be a self-reducible relation. There exists an FPRAS for approximating $|R(x)|$ if and only if there exists an FPAUS for sampling uniformly from $R(x)$.*

Proof of Theorem 4.4. The target uniform conditional distribution is defined as follows:

$$p(y_i \mid x, y_{<i}) := \frac{\text{EXT}_R(x, y_{<i}y_i)}{\sum_{u \in \Sigma} \text{EXT}_R(x, y_{<i}u)} \quad (y_i \in \Sigma). \quad (93)$$

Assume an FPRAS $\mathcal{A}(x, \varepsilon, \delta)$ exists for the self-reducible relation $|R(x)|$. By Proposition C.16, the existence of an FPRAS for $|R(x)|$ implies the existence of an FPRAS for the extension function EXT_R . We construct a CoT that samples $y \in R(x)$ by sequentially approximating these conditional probabilities. For each step $i \in \{1, \dots, m(n)\}$, the CoT computes $(1 \pm \varepsilon)$ -accurate estimates $\widehat{\text{EXT}}_R(x, y_{<i}u)$ for all $u \in \Sigma$ using $\mathcal{A}$, and induces the following distribution:

$$\pi(y_i \mid x, y_{<i}) := \frac{\widehat{\text{EXT}}_R(x, y_{<i}y_i)}{\sum_{u \in \Sigma} \widehat{\text{EXT}}_R(x, y_{<i}u)}. \quad (94)$$

Conditioned on the event that all estimates in Equation (94) are $(1 \pm \varepsilon)$ -accurate, the multiplicative error is bounded by:

$$\frac{1 - \varepsilon}{1 + \varepsilon} \leq \frac{\pi(y_i \mid x, y_{<i})}{p(y_i \mid x, y_{<i})} \leq \frac{1 + \varepsilon}{1 - \varepsilon}. \quad (95)$$

To ensure the cumulative approximation error remains within $(1 \pm \varepsilon')$, we set the local precision to $\varepsilon \leq \frac{\varepsilon'}{2 + \varepsilon'}$, which yields $\frac{1 + \varepsilon}{1 - \varepsilon} \leq 1 + \varepsilon'$ and $\frac{1 - \varepsilon}{1 + \varepsilon} \geq 1 - \varepsilon'$. To ensure the failure probability is at most δ' , we apply a union bound over the $m(n)$ generation steps and the $|\Sigma|$ calls per step. By setting the local confidence to $\delta \leq \frac{\delta'}{m(n)(|\Sigma| + 1)}$, the joint success event holds with probability at least $1 - \delta'$. Under this event, the CoT correctly simulates an FPRAS for $p(y_i \mid x, y_{<i})$ in total time $\text{poly}(n, 1/\varepsilon', \log(1/\delta'))$. Finally, since CoT can represent an FPRAS by Lemma 4.3, it satisfies the requirements for the construction.

On the other hand, we show that latent thought cannot compute such an approximation. Suppose, for contradiction, that the model could compute the conditional distribution $\pi(y_i \mid x, y_{<i})$ to within a $(1 \pm \varepsilon)$ relative error in a single step. We define the estimator for the total count $Z(x) = |R(x)|$ as:

$$\widehat{Z}(x) := \left(\prod_{i=1}^{m(n)} \pi(y_i \mid x, y_{<i}) \right)^{-1}. \quad (96)$$

Since the true distribution satisfies $p(y \mid x) = 1/|R(x)| = \prod_i p(y_i \mid x, y_{<i})$, the relative error of $\hat{Z}(x)$ is governed by the product of local errors:

$$(1 + \varepsilon)^{-m(n)} |R(x)| \leq Z(x) \leq (1 - \varepsilon)^{-m(n)} |R(x)|, \quad (97)$$

By setting $\varepsilon \leq \frac{\varepsilon'}{2m(n)}$, we apply Proposition C.14 and the properties of multiplicative error:

$$(1 + \varepsilon)^{-m(n)} \geq 1 - m(n)\varepsilon \geq 1 - \varepsilon'/2 > 1 - \varepsilon', \quad (98)$$

and for sufficiently small ε ,

$$(1 - \varepsilon)^{-m(n)} \leq 1 + 2m(n)\varepsilon \leq 1 + \varepsilon'. \quad (99)$$

Substituting these into Equation (96), we obtain:

$$(1 - \varepsilon')|R(x)| \leq \hat{Z}(x) \leq (1 + \varepsilon')|R(x)|. \quad (100)$$

This implies that if the model could compute π accurately, $\hat{Z}(x)$ would constitute an FPTAS for $|R(x)|$. However, under the standard complexity-theoretic assumption that $\text{FPTAS} \subsetneq \text{FPRAS}$ for self-reducible relations, this yields a contradiction. $\square$

D. Experimental Details

D.1. Fundamental Algorithmic Reasoning Tasks

D.1.1. TASK SETTINGS

Word Problem We define a sequence prediction task based on finite groups such as the symmetric group S_5 . Given a sequence of group elements of length k , the model is required to output the cumulative products obtained by scanning the sequence from left to right. Formally, for an input sequence $(g_1, g_2, \dots, g_k)$, the target sequence is $(g_1, g_1g_2, g_1g_2g_3, \dots, g_1g_2 \dots g_k)$. We follow the setting of (Merrill & Sabharwal, 2025b).

Connectivity To ensure that the reachability labels are approximately balanced, we generate undirected graphs according to the Erdős–Rényi model (Erdős & Renyi, 1959) $G(n, p)$, where n is the number of vertices and each possible edge is included independently with probability p . In the supercritical regime ($pn = c > 1$), a single “giant” connected component emerges, occupying a fraction $s \in (0, 1)$ of the vertices, which satisfies $s = 1 - e^{-cs}$. Consequently, the probability that two uniformly random vertices are both in this component—and hence mutually reachable—is approximately s^2 . To target a reachability probability of $1/2$, we set $s \approx \sqrt{1/2} \approx 0.707$, $c \approx \frac{-\ln(1-s)}{s} \approx 1.74$, and thus $p = \frac{c}{n} \approx \frac{1.7}{n}$. In practice, for each graph of size n we fix $p = 1.7/n$, which empirically yields $\Pr[\text{reachable}] \approx 50\%$ for $n \in [50, 100]$. We follow the encoding scheme of Sanford et al. (2024a). The input to the model is serialized as a flat token sequence consisting of three parts: $v_0 v_1 \dots v_{n-1} e_1 e_2 \dots e_m s, t$ where each vertex is denoted by a token v_i , each edge is represented as a pair “ u, v ” with $u < v$, and the final token “ s, t ” specifies the source–target pair for the reachability query.

Arithmetic Expression Evaluation Following (Feng et al., 2023), we generate expressions over integers modulo r using the four operations $+, -, \times, \div$, where multiplication and division are defined via precomputed modular tables. To guarantee that each expression evaluates to a specific target value, we grow expressions *backwards*: starting from a sampled number, we iteratively replace it with a binary sub-expression that preserves its value under modular arithmetic. Different from (Feng et al., 2023), we fix the modulus to $r = 3$, as our focus lies in evaluating the reasoning over expressions rather than exploring the properties of each modular arithmetic system.

Edit Distance The Edit Distance task requires computing the minimum number of edit operations needed to transform one string into another. The allowed operations are insertion, deletion, and replacement of a single character, and the objective is to predict the total edit distance given two input strings. To build the dataset, we follow (Feng et al., 2023). We first generate two strings over a randomly sampled alphabet. The first string has a fixed length, while the second string is produced in two possible ways: with probability 0.4, it is drawn as a random string of nearly the same length (within ± 3 characters), and with probability 0.6, it is derived from the first string by applying a given number of random edit operations. Each edit operation is chosen uniformly among deletion, replacement, and insertion. To avoid trivial cases, string pairs that are identical or whose lengths differ excessively are rejected and resampled. Finally, the shorter string is always placed first to maintain a consistent input format. An example instance in the format is shown below:

Table 3. Results for CoT on parallelizable tasks trained with uniform selection.

Task	n	Chain of Thought			
		8	16	32	64
Word Problem	64	0.8	0.8	0.8	100.0
Graph Connectivity	32	81.0	81.4	83.6	100.0
Arithmetic Evaluation	16	41.0	41.5	41.2	82.5
Edit Distance	16	69.2	70.3	82.6	94.8

s v d h s s e e ... v e | s h d s s s s ... e s e <sep> 20 Here, the two input strings are separated by the token “|”, “<sep>” marks the end of the inputs, and the final number “20” denotes the computed edit distance.

D.1.2. TRAINING CONFIGURATION

Configuration of chain of thought For CoT models, training is performed with supervision of step-by-step algorithms. **(1) Word problem:** for this task, the CoT algorithm proceeds by sequentially scanning the token sequence and producing at each prefix the evaluation result of the expression step by step. Thus, the overall length of the CoT sequence matches the input length. **(2) Graph connectivity:** Following Bavandpour et al. (2025), the algorithm sequence is simply the trace of a breadth-first search (BFS) starting from the source s . At each step, the model emits the incident edges of the currently expanded node in the order they are visited. The sequence terminates as soon as the target t is discovered. To implement this algorithm, we maintain a list (“scratchpad”) initialized with a dummy marker and the source, (N, s) . We iterate through this list from left to right (i.e., queue order). Whenever the current node u is expanded, we append to the end of the list all incident edges (u, v) for neighbors v , followed by a separator token (u, N) . **(3) Arithmetic expression evaluation:** Following (Feng et al., 2023), the CoT takes the fully expanded expression and repeatedly evaluates one innermost subexpression, writing down the simplified expression at each step until only a single numeral remains. For example, $2 * (0 + 1) / 2 \rightarrow 2 * 1 / 2 \rightarrow 2 / 2 \rightarrow 1$. The overall CoT sequence has quadratic length. **(4) Edit distance:** Following (Feng et al., 2023), the CoT algorithm outputs the DP table entries in the same order they are computed, i.e., row by row from top-left to bottom-right (topological order). This yields a quadratic number of steps in the input length.

Optimization and model details. We trained all models using the AdamW optimizer with a linear learning rate schedule. The initial learning rate was set to 1×10^{-4} with a weight decay of 0.01, and a batch size of 256. Training was continued until the training loss plateaued. For looped TFs, curriculum learning was applied to all tasks except edit distance: the input size was increased by 2 for the word problem task, and by 4 for the connectivity and arithmetic evaluation tasks. The model architecture was based on standard Transformers with an embedding dimension of 256. We used 4 attention heads, and varied the number of Transformer layers depending on the task: two layers for word problems, a single layer for connectivity, and time-modulated (Xu & Sato, 2025) model with a single layer for looped TF, to stabilize training, on both arithmetic evaluation and the edit distance task. For CoT, we use the same configuration of the Transformer block.

Uniform selection. To estimate a lower bound on the number of reasoning steps required by CoT for each task, we first follow the procedure introduced in prior work (Bavandpour et al., 2025). Specifically, given a complete CoT trajectory consisting of T intermediate reasoning steps, we construct shortened trajectories by uniformly selecting k step indices from $\{1, \dots, T\}$, i.e., by taking a uniformly spaced subsequence of the original trajectory. Only the selected intermediate steps are used, while the remaining steps are removed. We then evaluate task performance as a function of k , as shown in Table 3.

Stepwise internalization. We adopt stepwise internalization proposed by (Deng et al., 2024), a curriculum-based training procedure that gradually removes CoT tokens and encourages the model to internalize intermediate reasoning within its hidden states. Starting from a model trained on full CoT trajectories, we progressively truncate intermediate reasoning tokens according to a predefined schedule and finetune the model at each stage. Specifically, when the CoT length is greater than 128, we remove 16 tokens per stage until the remaining CoT length reaches 128. We then continue removing 8 tokens per stage until the CoT length is reduced to 8, training the model for 16 epochs at each stage. The results for each fundamental algorithmic reasoning task are shown in Fig. 10.

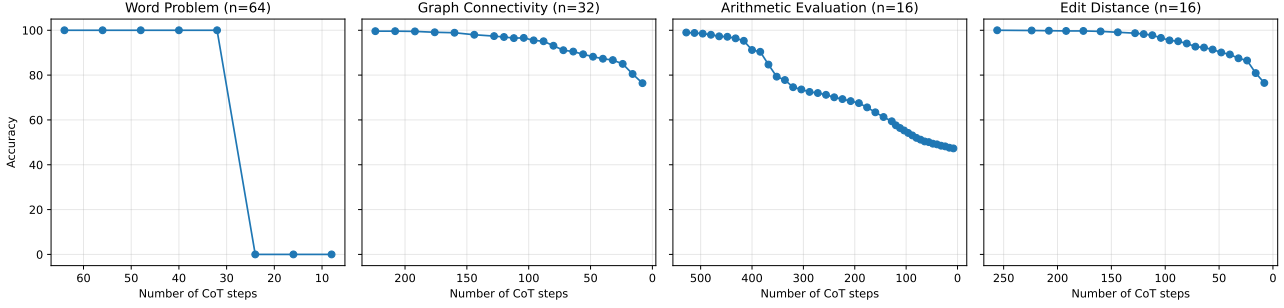


Figure 10. Accuracy as a function of the CoT steps during stepwise internalization.

D.2. Approximate Counting and Approximate Sampling

D.2.1. APPROXIMATE COUNTING OF DNF FORMULAS

To generate the dataset, we first construct a DNF formula F by sampling m clauses, each consisting of w distinct literals over n Boolean variables. Each literal is independently assigned to be either positive or negated. The formula is then serialized into a token sequence in which each clause is represented by its index together with variable–value pairs such as “2 = +1” or “4 = −1”. For looped TFs, we prepare 100,000 training samples and 1,000 test samples. For CoT models, we instead generate an online dataset with the following structure. To train the CoT models, we simulate a single trial of the randomized counting algorithm of Karp & Luby (1983). The sequence concatenates the serialized formula, the sampled clause, the full assignment, and the verification outcome, separated by `<sep>` tokens and terminated by `<eos>`. CoT model is trained in an autoregressive manner, where prediction targets are defined by shifting the token sequence while masking out the formula description. We trained the models using the AdamW optimizer with a linear learning rate schedule. The initial learning rate was set to 1×10^{-4} , with a weight decay of 0.01. We used a batch size of 256 (reduced to 32 for the 1000-loop setting) and trained for 10,000 iterations. For inference, we count the total number of iterations used for summing output tokens (steps) in CoT across trials, and the number of loop iterations in the looped TF.

D.2.2. APPROXIMATE SAMPLING OF GRAPH COLORINGS

We consider the problem of *approximately sampling* a proper k -coloring of a graph, also known as *almost-uniform generation*, a canonical randomized task closely related to #P-hard counting problems. Given an undirected graph $G = (V, E)$ with $n = |V|$ vertices and maximum degree Δ , a *proper k -coloring* is an assignment of colors from $\{1, \dots, k\}$ to vertices such that no adjacent vertices share the same color. Let $\Omega_k(G)$ denote the set of all proper k -colorings of G . We restrict attention to graphs of bounded degree and assume $k \geq 2\Delta + 1$. Under this condition, classical results in approximate counting show that the number of proper k -colorings admits a FPAUS (Jerrum, 1995). The approximation relies on Markov Chain Monte Carlo (MCMC) sampling using Glauber dynamics for graph colorings. Starting from an arbitrary proper coloring, the Markov chain repeatedly selects a vertex uniformly at random and proposes to recolor it with a randomly chosen color, accepting the update only if the resulting coloring remains proper. When $k \geq 2\Delta + 1$, this Markov chain is known to be rapidly mixing, converging to the uniform distribution over $\Omega_k(G)$ in polynomial time. For our experiments, we generate an undirected Erdős–Rényi random graph, where each edge is included independently with probability $p = \frac{1.7}{n}$, using a fixed random seed for reproducibility. For simplicity and ease of analysis, we focus on small graphs with $n = 3$ and set the number of colors to $k = 5$.

Each sample is generated by running T steps of Glauber dynamics on the space $\Omega_k(G)$ of proper k -colorings. Starting from a greedy proper initialization, at each step we uniformly select a vertex and a color; the recoloring is accepted if and only if it preserves properness. The final state of the Markov chain is treated as an approximate sample from the uniform distribution over $\Omega_k(G)$. In addition to the final coloring, we optionally record the entire sequence of proposals and accept/reject outcomes for CoT and use it as supervision, which is not included for latent thought. To train sequence models, we serialize the graph structure, the initial coloring, and either the MCMC history or the final coloring into a single token sequence. We evaluate the distribution of solutions generated by the model rather than only solution accuracy. Given an input x , we draw N independent samples from the model using ancestral decoding. Generation continues until an end-of-sequence (EOS) token is produced, and from each generated sequence we extract the final n tokens immediately

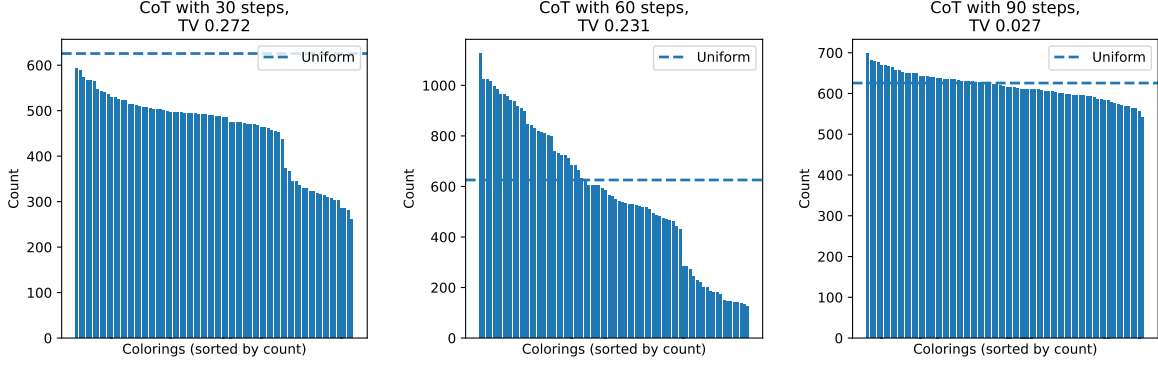


Figure 11. Output distributions of CoT compared against the uniform target distribution.

preceding the first EOS token, which encode a candidate solution. The resulting samples define an empirical distribution $\hat{P}$ over generated solutions via normalized occurrence counts. For each instance, the task provides an exact enumeration $\mathcal{Y}$ of all valid solutions. We define the reference distribution P_{true} as the uniform distribution over this solution set, assigning probability $1/|\mathcal{Y}|$ to each $y \in \mathcal{Y}$ and zero otherwise. We quantify the discrepancy between the empirical distribution $\hat{P}$ and the uniform reference distribution using the total variation distance $\frac{1}{2} \sum_{y \in \mathcal{Y} \cup \hat{\mathcal{Y}}} |\hat{P}(y) - P_{\text{true}}(y)|$. We trained the models using the AdamW optimizer with a linear learning rate schedule. The initial learning rate was set to 1×10^{-4} , with a weight decay of 0.01. We used a batch size of 256 and trained for 5,000 iterations. For inference, we measure the average number of steps (loops) per generation. We generate $N = 50,000$ samples for CoT and $N = 10,000$ samples for looped TFs. The resulting histograms of CoT outputs over the target support are shown in Figure 11.